

基于“径切向移位迭代模型”的无人机纯方位无源定位

摘 要

本文基于纯方位无源定位的假设，解决了圆形与锥形无人机编队的定位和编队调整问题，上述所有算法精度均可达机器精度（ 10^{-11} ）。

针对问题一（1），为实现圆形编队下，三架无人机发送信号时，其余无人机的自我定位任务，在发射信号的无人机位置无偏差且编号已知的前提下，我们首先通过已知编号将接收角和发射信号的无人机进行对应，然后在极坐标系下的三角形中多次使用正弦定理即可求解其准确坐标。在理论上，我们可以遍历每一种情况，并根据不同几何形状来准确计算。为了简化模型，我们首先利用圆形的对称性简化发射机的组合方式（取 4 种即可），然后通过定义有向角，拓展极坐标的定义，划分平面区域等操作，构建了不同区域下的统一数学模型。将被动定位的无人机在极坐标系下的位置 $P(\rho_0, \theta_0)$ 表示如下： $\theta_0 = \arctan \left[\frac{\sin\beta_1 \sin(\beta_2 + \alpha) - \sin\beta_1 \sin\beta_2}{\sin\beta_1 \cos(\beta_2 + \alpha) + \sin\beta_2 \cos\beta_1} \right]$, $\alpha = k \cdot \frac{2\pi}{9}$, $k = 1, 2, 3, 4$, $\rho_0 = \frac{\sin(\theta_0 + \beta_1)}{\sin\beta_1} R$ ，其中 β_1 和 β_2 为转化得到的有向角， α 和 k 分别代表第三架无人机的极角和编号。经 matlab 仿真，模型误差可达机器精度（ 10^{-11} ）。

针对问题一（2），由于圆形队列具有轴对称性，因此仅用两架无人机无法实现纯方位无源定位。因此我们首先假设只需新增一架无人机发射信号即可实现定位。由于未知的发射信号的无人机位置和被动定位的无人机位置组合情况较少，故可枚举。我们构建扰动模型，在“接收角域”对每一种组合情况进行 50000 次的蒙特卡洛模拟，发现接收角在任何情况均不存在混叠的现象。然后我们通过多分类算法实现了对收到的“接收角”进行有效分类，从而精准定位出未知位置的发射信号的无人机。至此，模型与问题一（1）同，代入模型可求解。

针对问题一（3），题目要求无人机仅通过接收到的方向信息来进行编队调整，且只有坐标原点是已知并不变的。我们充分利用了圆的几何性质，提出了一种仅根据方向信息可实现定位的径切向移位迭代算法，在第 300 次的迭代时，圆周上的无人机已均匀分布在新圆上，且径/切向误差均已达机器精度（ 10^{-11} ）。为了对比此算法的性能，我们增设了一组将队列整体与各自的最优位置间距之和设为损失函数的编队调整算法。结果证明，后者的总体无人机移动路径最小且误差减少更快，但从算法收敛性、模型稳定性来看，前者的算法均以绝对优势领先。

针对问题二，为实现锥形无人机编队调整，我们分别构建了定位模型和调整模型。前者沿用问题一（1）中“三点定位”的思想，后者是问题一（3）中“径切向移位迭代算法模型”的迁移。

综上所述，将本论文工作总结如下：利用正弦定理计算出了“三架无人机编队纯方位定位模型”；设计出了基于扰动模型和蒙特卡洛的多分类模型；利用圆形的几何特性建立出了仅靠角度信息即可校准位置的“径切向移位迭代模型”；并将上述算法成功迁移到了锥形编队中。

关键词：纯方向无源定位，无人机编队控制，蒙特卡洛算法，Nelder-Mead 单纯形算法，多目标优化

一、问题重述

1.1 问题的背景

无人机集群在遂行编队飞行中为避免外界干扰，需要尽量保持电磁静默。为保持编队队形，采用纯方位无源定位的方法调整无人机的位置，即由编队中某几架无人机发送信号，其余无人机接收信号并提取方向信息定位，以此来调整位置。

1.2 问题的重述

问题一：10架无人机在同一高度组成圆形编队，1架无人机（FY00）位于圆心，另9架无人机（FY01~FY09）均匀分布在该圆的圆周上。

（1）位于圆心的无人机和编队中另2架无人机发射信号，若发射信号的无人机位置无偏差且编号已知，建立其余位置略有偏差的被动接收信号无人机的定位模型。

（2）某位置略有偏差的无人机接收到编号为FY00和FY01的无人机发射的信号，若发射信号的无人机位置无偏差，还需要几架无人机发射信号，可以实现无人机有效定位。

（3）利用表一给出的无人机的初始位置，在无人机位置略有偏差时，仅根据接收到的方向信息给出具体的调整方案，即经过多次调整，每次选择编号为FY00的无人机和圆周上最多3架无人机发射信号，使得9架无人机均匀分布在圆周上。

问题二：仍考虑纯方位无源定位的情形，设计锥形编队无人机的位置调整方法。

二、问题分析

2.1 问题一的分析

问题一给定了编队队形，从中我们可以得到不同编号的无人机在初始情况下的相对位置情况。圆形的队形中还包含了几何对称和调整前后的队形相似的信息。

2.1.1 问题一的（1）的分析

在问题一的（1）中，由于定位方式为纯方位无源定位，因此在三架发射信号的无人机编号已知的情况下，作为被动接收信号的无人机，能够接收到的信息为“发射信号的三架无人机的编号”与“三个接收角的大小”。虽然接收到的信息中并没有“接收角与发射信号的无人机间的对应关系”，但是在编队队形已知，且无人机扰动微小的情况下，可以根据信号来自的方位和接收角的大小做出判断，以此来得到“接收角与发射信号的无人机间的对应关系”。

以位于圆心处的无人机FY00为极点，以由FY00指向另一发射信号的无人机（以FY01为例）的射线为极轴建立极坐标系。可以利用圆的几何对称性和被动接收信号的无人机相对于极轴的对称性简化问题。第三架发射信号的无人机位置只需考虑FY02、FY03、FY04、FY05四种情况即可。

基于上述对方向信息的分析和模型的简化，我们通过角的有向化、极坐标的拓展定义和不同区域内接收角与有向角的对应关系，结合三角形中的正弦定理，计算出被动接收信号的无人机的位置的统一公式，以此建立被动接收信号无人机的定位模型。

为了验证模型的正确性，我们遍历所有发射信号的无人机相对位置的可能情况，将该位置与带有初始扰动的位置进行比较，成功验证了模型的正确性。

2.1.2 问题一的（2）的分析

问题一的（2）相对于（1）中主要的区别是有若干发射信号的无人机的编号未知，需要去求解还需要几架无人机发射信号才能实现无人机的有效定位。基于背景知识，可知发射信号的无人机越少越好。同时由于在（1）中已经建立了在三架发射信号的无人机编号已知的情况下的定位模型，所以我们优先考虑还需要一架无人机即可实现有效定位。由于编队队形已知，其中包含了初始情况下的位置信息，如果在微小扰动的前提下，被动接收信号的无人机可以通过接收到的角度信息判断出第三架无人机的编号，就把问题迁移到了（1）的模型中。

基于上述分析，我们进行了考虑扰动模型下的蒙特卡洛模拟分析，根据结果可知，被动接收信号的无人机确实可以通过接收到的角度信息直接判断出第三架无人机的编号。至此，已经与（1）中建立的模型达成了一致。

为了检验蒙特卡洛模拟的有效性，我们通过批量随机生成数据和多分类算法，进行了模型验证，结果证明模型有效。

2.1.3 问题一的（3）的分析

问题一的（3）要求无人机仅通过接收到的方向信息来进行编队调整，且只有坐标原点是已知并不变的。因此我们将问题拆解为：①选择最适合作为发射信号的无人机；②圆周上的无人机能够通过接收到的方向信息校验自己是否需要调整；③需要调整的无人机通过径向/切向移动到合适位置（通过接收到的方向信息判别是否合适）；④以一定的次序对圆周上所有无人机的位置迭代更新；⑤圆周上的无人机能够在有限次迭代过程中收敛到稳定的等间距的圆周分布。

由于圆周上的每架无人机只能得知自己的编号与接收到的方向信息，即知道自己所处的相对位置，并在扰动允许的情况下根据方向信息还原出圆周上发射信号的无人机的编号。据此，我们需要充分利用圆的几何特性，设计迭代过程：按逆时针顺序依次选取圆周上相邻的两架无人机与坐标原点处的无人机发射信号，针对每个发射组合，我们设定合适的阈值使圆周上的这两架无人机关于坐标原点“对称”位置的无人机能够判断自己需要调整，并进行径向/切向调整。调整终止的条件是接收角中的最小两角之和为 20° 或之差为 0° 。经 matlab 仿真验证，模型有效且可以以较快速度收敛至新的圆周上。

2.2 问题二的分析

为了实现锥形编队队形的调整，我们依次建立了定位模型和基于定位模型的调整模型。在该模型的建立中，我们主要的工作是进行问题一中模型的迁移与适配，体现了模型的可移植性。其中定位模型沿用问题一（1）中“三点定位”的思想，但不局限于等腰三角形，对任何形状的三角形均适用；其中调整模型继承了问题一（3）中的“径切向移位迭代算法”的核心思想，并将其进行普适推广。

三、模型假设

1. 被动接收信号的无人机得到的接收角为劣角(指大于 0° 而小于 180° 的角)。
2. 被动定位的无人机在定位中知道自己的编号。
3. 微小扰动不会影响无人机的相对位置。

四、符号说明

符号	说明	单位
$\alpha_k, k = 1, 2, 3$	圆形编队中被动接收信号的无人机的接收角	$^{\circ}$ 或 rad
$\beta_k, k = 1, 2, 3$	圆形编队中接收角的有向角定义	$^{\circ}$ 或 rad
δ_R	圆形编队中无人机扰动的径向变化	m
δ_{θ}	圆形编队中无人机扰动的切向变化	$^{\circ}$ 或 rad
γ	圆形编队中被动接收信号的无人机的极角	$^{\circ}$ 或 rad
E	判断是否进行调整的阈值	$^{\circ}$ 或 rad
F_{move}	需要移动的无人机	\
R_{best}	调整后圆形编队的最优半径	m
θ_{best}	调整后圆形编队的最优起始角度	$^{\circ}$ 或 rad
λ	位移延迟因子	\
λ_{const}	常数衰减的位移延迟因子	\
$\lambda_{decrease}$	指数衰减的位移延迟因子	\
V_{loss}	迭代过程中与最优位置的距离之和的均值	\
$V_{distance}$	迭代过程中无人机移动距离之和	m
V_{speed}	模型稳定的速度	\
$\alpha'_k, k = 1, 2, 3$	锥形编队中被动接收信号的无人机的接收角	$^{\circ}$ 或 rad
$\beta'_k, k = 1, 2, 3$	锥形编队中接收角的有向角定义	$^{\circ}$ 或 rad
δ'_R	锥形编队中无人机扰动的径向变化	m
δ'_{θ}	锥形编队中无人机扰动的切向变化	$^{\circ}$ 或 rad
γ'	锥形编队中被动接收信号的无人机的极角	$^{\circ}$ 或 rad

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 问题一的（1）

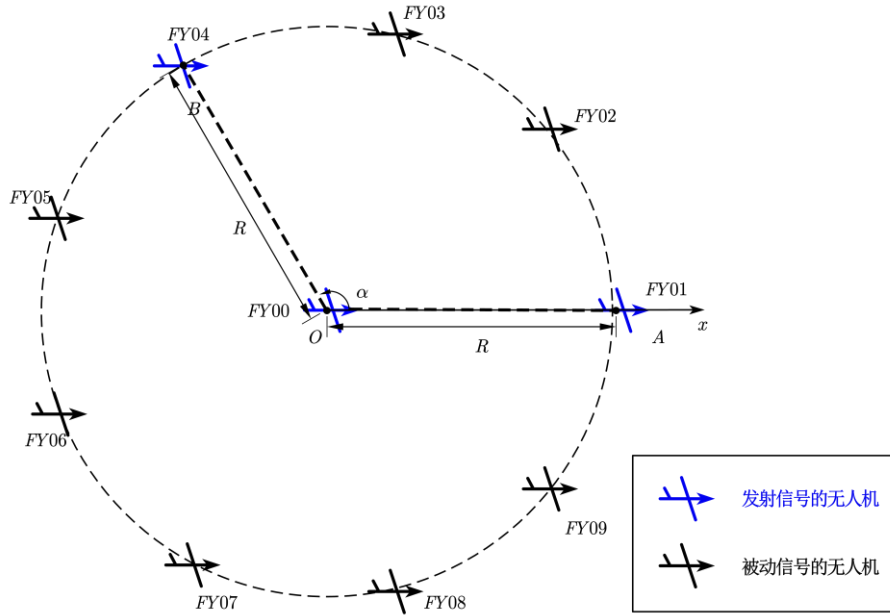
（一）模型的建立

1. 纯方向无源定位的关键：接收信号的无人机与发射信号的无人机之间的相对位置。为表述清晰，不妨假定FY01为第二架发射信号的无人机来进行研究，以此建立普适模型。

2. 由于发射信号的无人机编号均已知，且无人机位置扰动微小，不会引起被动接收信号的无人机相对于发射机的位置改变，所以可以得到接收角与发射信号的无人机间的对应关系。

3. 建立极坐标系和符号说明：规定FY00为O，FY01为A，第三架发射信号的无人机为B，以O为坐标原点，以由O指向A的射线Ox作为极轴建立极坐标系，规定极角逆时针方向为正，同时进行拓展定义，允许极径 $\rho < 0$ ，极角 θ 为任意值。示意图

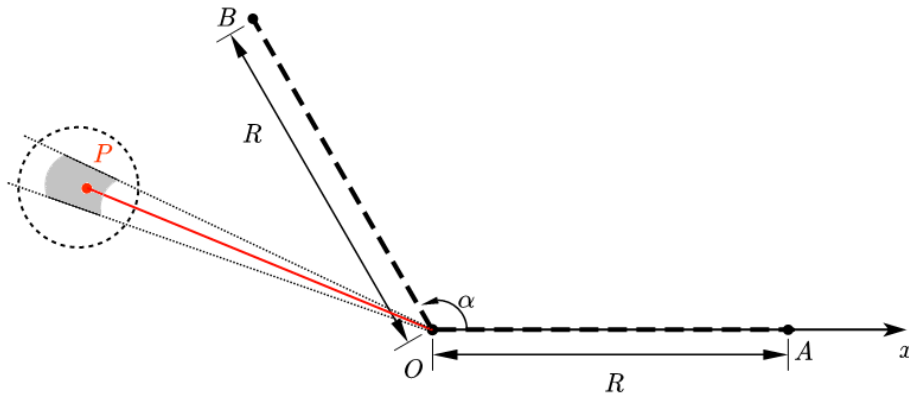
如图表 1 所示（以FY04为第三架发射信号的无人机为例）。设圆形编队的半径为 R 。以 O 为顶点且由 A 逆时针指向 B 的角度为 α 。



图表 1 圆形无人机编号的极坐标系

4. 根据对称性简化模型：由于圆周上共有九架无人机，且以FY00和FY01所在直线呈轴对称关系，因此第三架发射信号的无人机位置只需考虑FY02、FY03、FY04、FY05四种情况，根据 B 的位置可得 $\alpha = k \cdot \frac{2\pi}{9}$, $k = 1, 2, 3, 4$ 。

5. 构建扰动模型：将微小扰动导致被动定位的无人机的位置变化分解为径向变化 δ_R 和切向变化 δ_θ 。根据问题一的（3）中的表格信息，取最小值为波动下限，最大值为波动上限，规定变化范围为 $\delta_R \in [-0.02R, 0.12R]$, $\delta_\theta \in [-0.30^\circ, 0.30^\circ]$ ，且扰动在范围内均匀分布，作出扰动示意图如图表 2 所示。



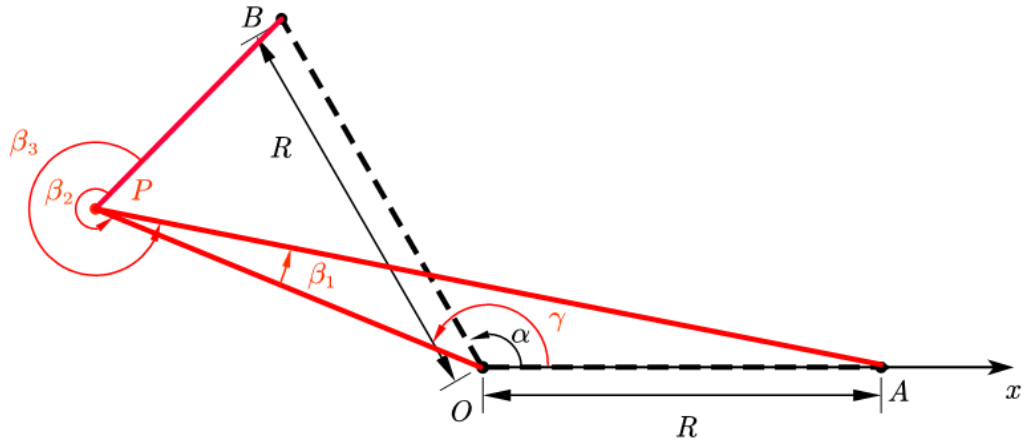
图表 2 圆形无人机编队的扰动模型

6. 建立圆形无人机编队的纯方位无源定位模型（发射信号的无人机共三架且编

号已知)：设被动定位无人机的位置为 P 。为了统一模型的最终表达式，我们使用有向角，并将有向角定义如下：

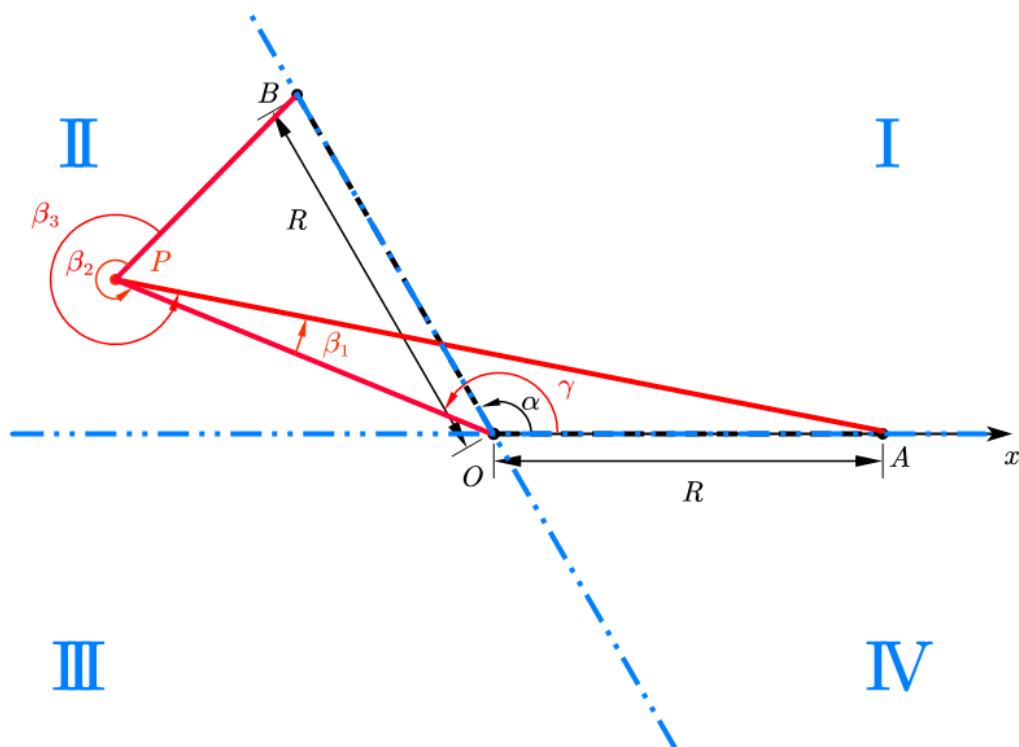
- a. 以 O 为顶点且由 A 逆时针指向 P 的角度为 γ 。
- b. 以 P 为顶点且由 O 逆时针指向 A 的角度为 β_1 。
- c. 以 P 为顶点且由 B 逆时针指向 O 的角度为 β_2 。
- d. 以 P 为顶点且由 B 逆时针指向 A 的角度为 β_3 。

在此定义下， $\beta_k, k = 1, 2, 3$ 角间有确定的代数关系，即 $\beta_3 = \beta_1 + \beta_2$ 恒成立。由于发射信号的无人机编号均已知且扰动微小，不会引起被动接收信号的无人机相对于发射机的位置改变，所以可根据相对位置和已知的编号来构建信号与发射机的对应关系，以此测量所得劣角为： $\angle OPA = \alpha_1, \angle BPO = \alpha_2, \angle APB = \alpha_3$ 。建立纯方位无源定位模型如图表 3 所示。



图表 3 圆形无人机编队中的定位模型

7. 构建接收角 α_k 与有向角 β_k 的对应关系（其中 $k = 1, 2, 3$ ）：由于我们在模型假设中设定接收角为劣角 α_k ，这与模型计算中使用的有向角 β_k 不是简单的相等关系。所以我们将以 OA 和 OB 所在直线为划分线，将平面划分为四个区域 I, II, III, IV 如图表 4 所示。



图表 4 分区域的圆形无人机编队中的定位模型

在不同区域中，接收角（劣角）和有向角的对应关系如表格 1 所示：

表格 1 圆形无人机编队接收角（劣角）和有向角的对应关系

四个区域	对应关系
第一区域 (I)	$\beta_1 = \alpha_1, \beta_2 = \alpha_2, \beta_3 = \beta_1 + \beta_2 = \alpha_1 + \alpha_2$
第二区域 (II)	$\beta_1 = \alpha_1, \beta_2 = 2\pi - \alpha_2, \beta_3 = \beta_1 + \beta_2 = 2\pi + \alpha_1 - \alpha_2$
第三区域 (III)	$\beta_1 = 2\pi - \alpha_1, \beta_2 = 2\pi - \alpha_2, \beta_3 = \beta_1 + \beta_2 = 4\pi - \alpha_1 - \alpha_2$
第四区域 (IV)	$\beta_1 = 2\pi - \alpha_1, \beta_2 = \alpha_2, \beta_3 = \beta_1 + \beta_2 = 2\pi - \alpha_1 + \alpha_2$

（二）模型的求解

利用正弦定理计算被动接收信号的无人机位置：通过在 $\triangle OAP$ 和 $\triangle OBP$ 中分别使用正弦定理构造二元方程求解两个未知量： OP 与 γ ，即被动接收信号的无人机的极坐标位置。

1. 接收角转化为有向角：在得到接收角 $\alpha_1, \alpha_2, \alpha_3$ 后，根据已知编号，利用被动定位的无人机与发射信号的无人机的相对位置，确定被动定位的无人机的位置所在区域，通过表 1 中接收角和有向角的对应关系计算出有向角 $\beta_1, \beta_2, \beta_3$ 。

2. 计算无人机 P 在极坐标系内的极角 γ :

a. 在 $\triangle OAP$ 中有:

$$\frac{OA}{\sin \beta_1} = \frac{OP}{\sin(\pi - \gamma - \beta_1)} \Rightarrow \frac{R}{\sin \beta_1} = \frac{OP}{\sin(\gamma + \beta_1)} \quad (1)$$

b. 在 $\triangle OBP$ 中有:

$$\frac{OB}{\sin(2\pi - \beta_2)} = \frac{OP}{\sin[\pi - (\gamma - \alpha) - (2\pi - \beta_2)]} \Rightarrow \frac{R}{\sin \beta_2} = \frac{OP}{\sin(\alpha + \beta_2 - \gamma)} \quad (2)$$

c. 由①式除以②式得到:

$$\frac{\sin \beta_2}{\sin \beta_1} = \frac{\sin(\alpha + \beta_2 - \gamma)}{\sin(\gamma + \beta_1)} = \frac{\sin(\alpha + \beta_2) \cos \gamma - \cos(\alpha + \beta_2) \sin \gamma}{\sin \gamma \cos \beta_1 + \cos \gamma \sin \beta_1} \quad (3)$$

因为扰动很微小, 所以 γ 不可能为 $\frac{\pi}{2} + n\pi, n \in \mathbb{Z}$, $\frac{\pi}{2} + n\pi, n \in \mathbb{Z}$, 一定可以化简为

$$\tan \gamma = \frac{\sin \beta_1 \sin(\beta_2 + \alpha) - \sin \beta_1 \sin \beta_2}{\sin \beta_1 \cos(\beta_2 + \alpha) + \sin \beta_2 \cos \beta_1} \quad (4)$$

由于对极坐标进行了拓展定义, $(\rho, \theta), (-\rho, \theta + \pi + 2nk), (\rho, \theta + 2nk), n \in \mathbb{Z}$ 表示同一个位置, 所以不需要对 γ 的范围进行讨论, 因此将 γ 表示为

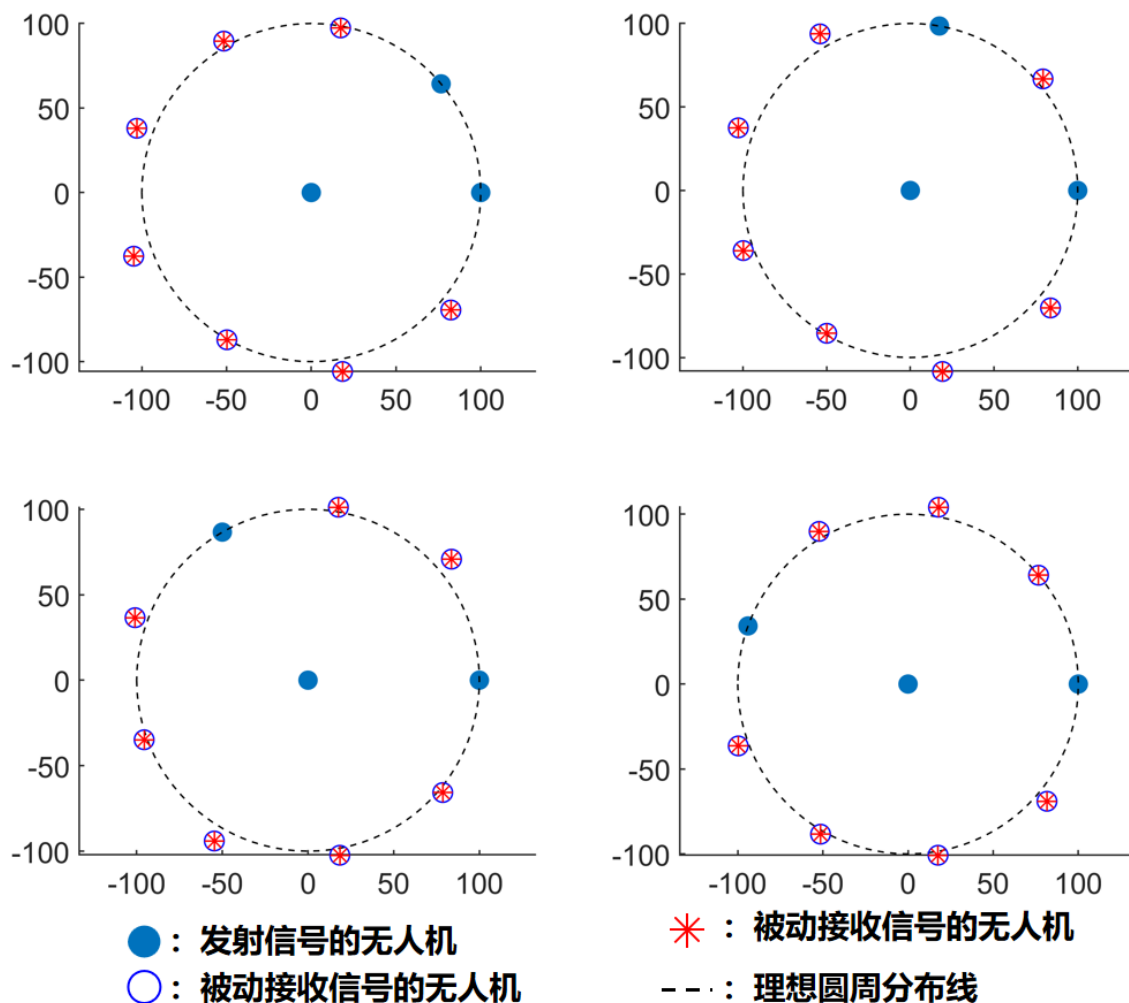
$$\gamma = \arctan\left[\frac{\sin \beta_1 \sin(\beta_2 + \alpha) - \sin \beta_1 \sin \beta_2}{\sin \beta_1 \cos(\beta_2 + \alpha) + \sin \beta_2 \cos \beta_1}\right] \quad (5)$$

其中 $\alpha = k \cdot \frac{2\pi}{9}, k = 1, 2, 3, 4$ 。

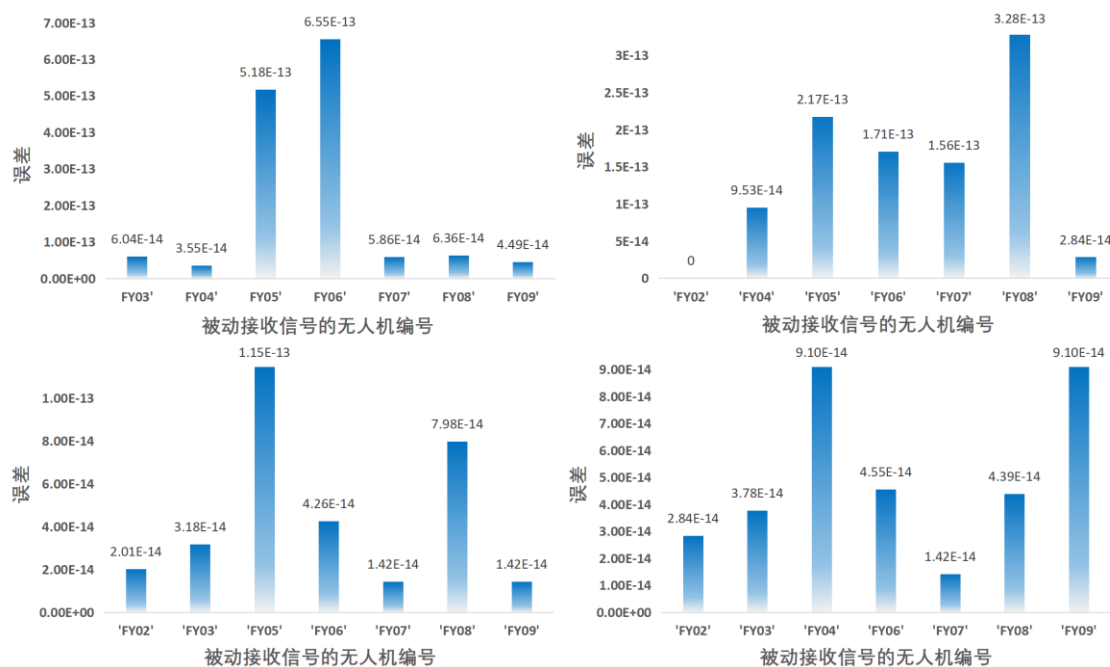
3. 计算无人机 P 在极坐标系内的极径 OP : 根据式(1)可得 $OP = \frac{\sin(\gamma + \beta_1)}{\sin \beta_1} R$, 即解得 P 在极坐标下的位置为 $P\left(\frac{\sin(\gamma + \beta_1)}{\sin \beta_1} R, \gamma\right)$ 。

(三) 模型的验证

由圆的对称性不妨选定 $FY00, FY01$ 为发射信号的无人机, 并将第三架无人机依次选定为 $FY02, FY03, FY04, FY05$ 进行模型验证 (见图表 5), 求得其余无人机位置, 我们将该位置和带有初始扰动的位置进行比较, 误差见图表 6, 均小于等于 10^{-13} 的量级。故模型验证成功。结果数据见附录 3_表格 1。



图表 5 圆形无人机编队定位模型的验证仿真图



图表 6 圆形无人机编队定位模型验证仿真图的误差

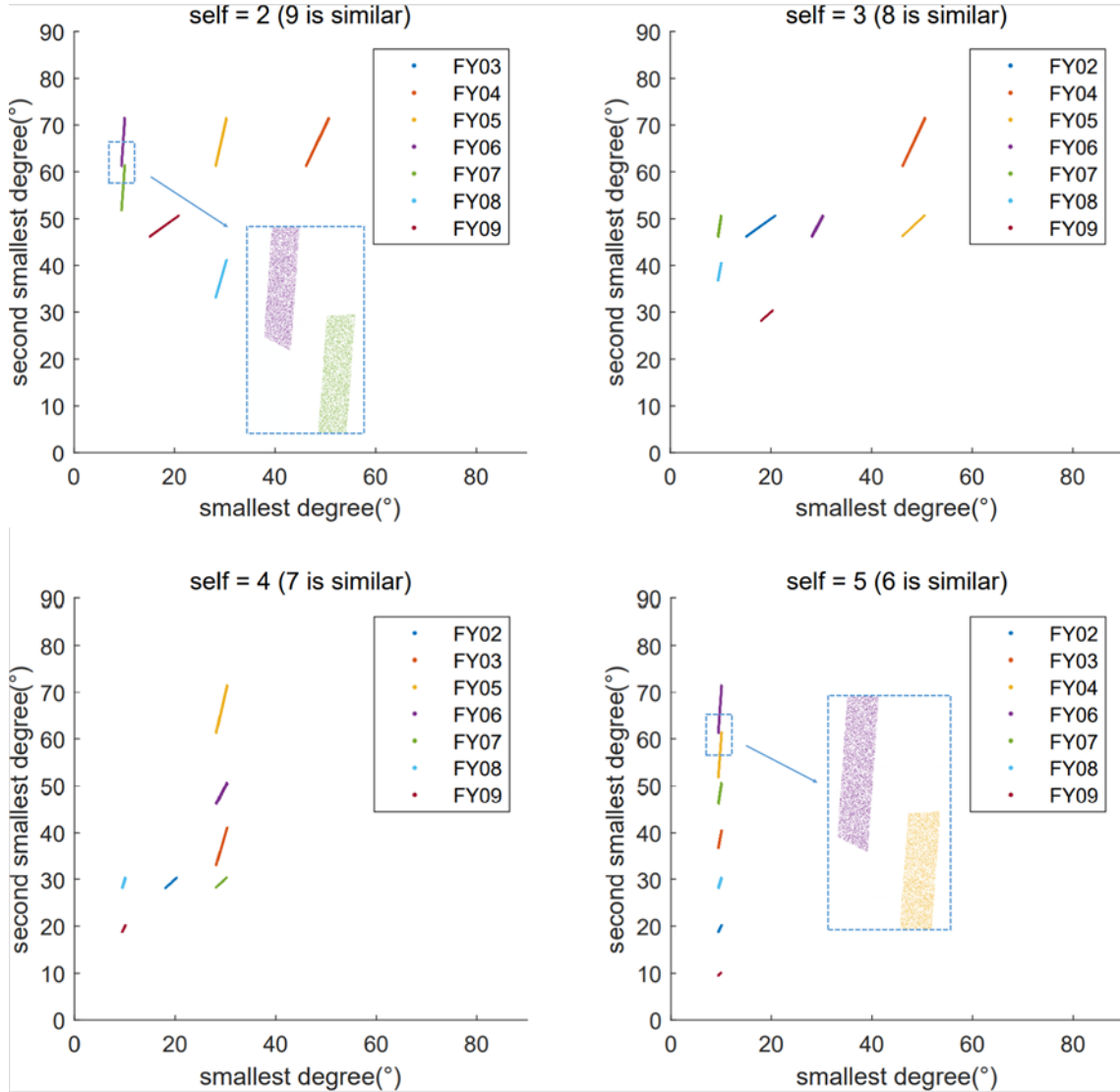
5.1.2 问题一的（2）

（一）模型的建立

根据背景知识，为保持电磁静默，发射信号的无人机越少越好。所以先假设还需 0 架，也就是在仅由编号为FY00和FY01的无人机发送信号的情况下，可以实现无人机的有效定位。然而这种情况显然不成立，例如以FY02和FY09作为接收信号的无人机，他们接收到的信息完全一致，所以不能实现无人机的有效定位。

然后我们假设除FY00和FY01外，还需要一架发射信号的无人机即可实现其余无人机的有效定位。由于圆周上无人机对FY00和FY01所在的直线呈轴对称性，则只需设被动定位的无人机编号为FY0Z，且 $Z = 2, 3, 4, 5$ 。设第三架发射信号的无人机为FY0W，其中 $W = 2, 3, \dots, 8, 9, W \neq Z$ 。

在考虑扰动模型的情况下，即在径向扰动 $\delta_R \in [-0.02R, 0.12R]$ ，切向扰动 $\delta_\theta \in [-0.30^\circ, 0.30^\circ]$ 的均匀分布模型下，我们枚举每个被动接收信号的无人机和每个可能的第三架发射信号的无人机的所有组合。针对每种组合，以较小的两个接收角为变量作图，采用蒙特卡洛方法模拟 50000 次，结果如图表 7 所示。



图表 7 考虑扰动模型的蒙特卡洛模拟结果

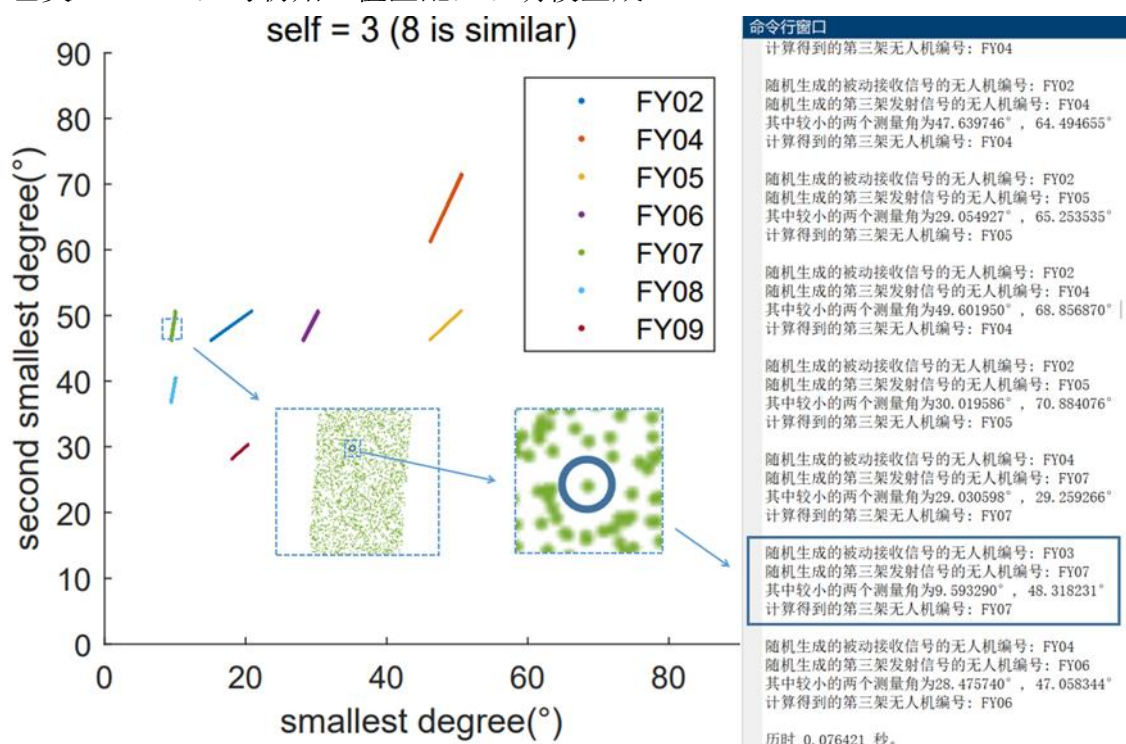
据此可以根据收到的三个接收角 $\alpha_1, \alpha_2, \alpha_3$ 的信息，通过多分类算法将其对应到不

同的类别中，即可还原第三架发射信号的无人机的位置。至此，我们将问题转化为问题一的（1），即“圆形编队下，三架无人机发送信号时，其余无人机的自我定位任务”，故代入问题一的（1）的定位模型计算即可。

综上所述，除FY00和FY01外，还需要一架发射信号的无人机才能实现其余无人机的有效定位。

（二）模型的验证

1. 随机生成一个2~9的数Z，作为被动接收信号的无人机的编号FY0Z。
2. 随机生成一个2~9且与第1步中的Z不同的数W，作为第三架发射信号的无人机的编号FY0W。
3. 在考虑有一架发射信号的无人机编号位置的情况下，得到方向信息，即三个接收角 $\alpha_1, \alpha_2, \alpha_3$ 。
4. 以其中较小的两个角为关键字生成坐标，进行多分类：计算该点与蒙特卡洛算法模拟出的350000个点之间的距离，找出距离最近的点，其所属的类别即为所要求的FY0W。
5. 如图表8所示，我们以Z=3, W=7为例，图中蓝框内的数据点成功被归类为绿色类（FY07），与初始W值匹配，证明模型成立。



图表 8 基于扰动模型和蒙特卡洛的多分类算法

5.1.3 问题一的（3）

由于圆周上无人机仅能通过接收到的方向信息调整自身位置，我们在充分考虑了圆形无人机编队的几何特性后，建模如下。

（一）模型的建立

1. 规定圆周范围上的无人机调整模式：径向调整或切向调整。
2. 迭代过程：

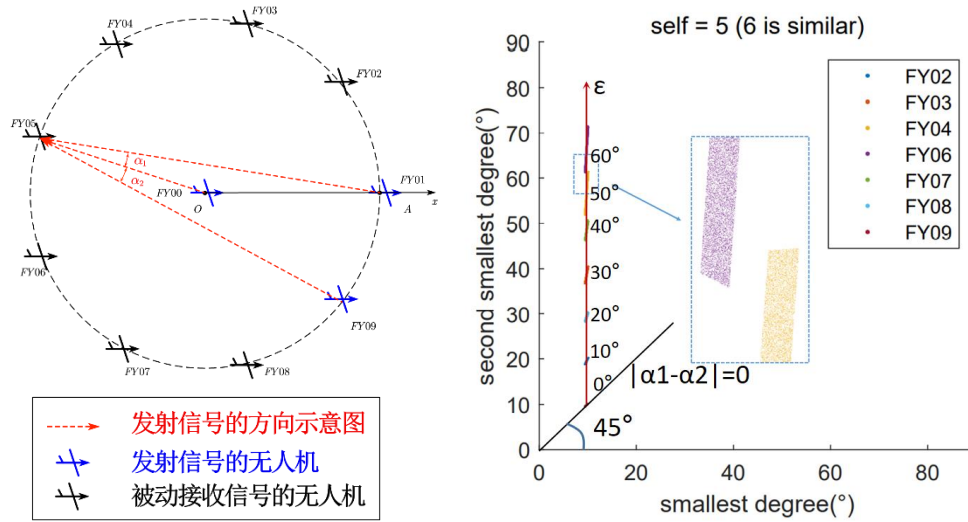
- a. 选取发射信号的无人机：随机选取圆周上相邻的两架无人机 F_1 与 F_2 和坐标原点处的无人机 F_0 作为发射信号的无人机，向四周辐射信号。
- b. 锁定需要进行调整位置的无人机：在径向扰动 $\delta_R \in [-0.02R, 0.12R]$ ，切向扰动 $\delta_\theta \in [-0.30^\circ, 0.30^\circ]$ 的前提下，其余各架被动接收信号的无人机判断接收角 $\alpha_k, k = 1, 2, 3$ 中的两个较小角的大小之差的绝对值是否小于阈值 E 。我们根据扰动信息和无人机的拓扑关系可以设置合适的阈值 E 使仅有一架被动接收信号的无人机满足条件，且该无人机即为需要调整位置的无人机，也即无人机 F_1 和 F_2 关于 F_0 “对称”的无人机，设该无人机为 F_{move} 。
- c. 定义无人机的径向调整：定义无人机 F_{move} 在径向方向上移动，直至其接收到的两个较小的角度信息之和为 20° 。
- d. 完成所有无人机的径向调整：按逆时针顺序依次选取相邻的两架无人机与无人机 F_0 作为发射信息的无人机，并在每种发射组合的情况下执行 b 步骤和 c 步骤，直至圆周上所有的无人机完成一次径向调整。
- e. 定义无人机的切向调整：定义无人机 F_{move} 在切向方向上移动，直至其接收到的两个较小的角度信息之差为 0° 。
- f. 完成所有无人机的切向调整：按逆时针顺序依次选取相邻的两架无人机与无人机 F_0 作为发射信息的无人机，并在每种发射组合的情况下执行 b 步骤和 e 步骤，直至圆周上所有的无人机完成一次切向调整。
- g. 不断循环步骤 d 和步骤 f。

（二）模型的求解

1. E 的求解：我们采用蒙特卡洛算法进行求解。如图表 9 所示，设FY09

FY00、FY01作为发射信号的无人机，作为被FY05选中的最优调整的无人机，由于 E 被定义为接收角中两个最小角的大小之差的绝对值。所以令 $|\alpha_1 - \alpha_2|$ 小于等于 E ，但其余各个无人机接收到的 $|\alpha_1 - \alpha_2|$ 应大于 E 。如图表 10 所示（本图取自问题一（2）中的蒙特卡洛结果，定义详见上文），我们做出函数 $|\alpha_1 - \alpha_2| = 0$ ，并以FY09作为误差线 ε 的坐标原点，依次可以写出各个无人机的误差 ε ，在

FY00、FY01、FY05已知的情况下，为了区分FY09（此处与FY05是否能自知为最优无人机等价），从图中可以直观看到，将 E 取在 $0.5^\circ \sim 9^\circ$ 中均可。

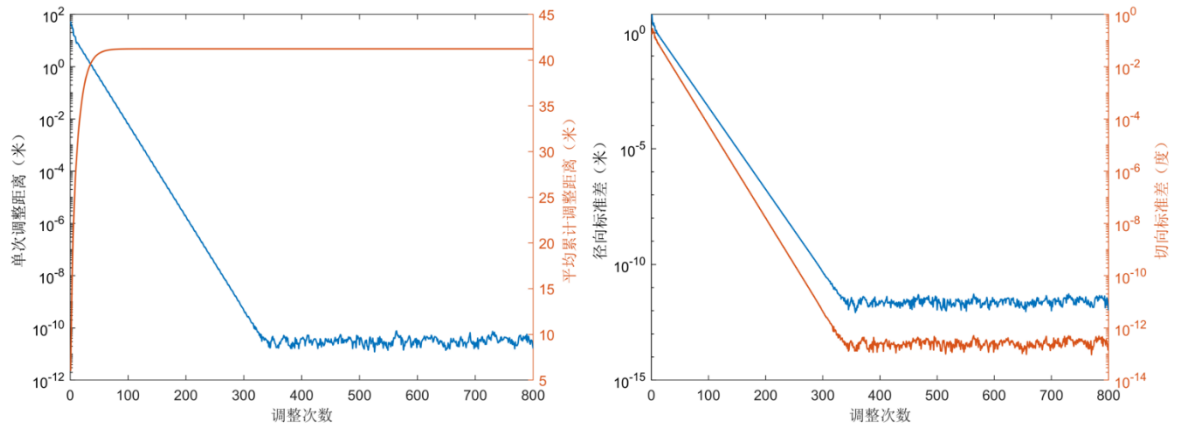


图表 9 发射信号与被动接收信号的无人机位置（左）

图表 10 发射信号与被动接收信号的无人机位置（右）

2. 迭代仿真：按照迭代过程执行 800 次迭代，迭代结果如图表 11 径切向移位迭代模型的迭代结果示意图所示。

3. 算法具有收敛性：我们发现相邻无人机的径向距离和切向距离误差均在 350 次迭代后达到机器精度 (10^{-11})，表明圆周上各个无人机均匀的落在了以 r 为半径的圆上。通过单次调整距离也达到机器精度，进一步证明了该算法的收敛性。



图表 11 径切向移位迭代模型的迭代结果示意图

5.1.4 问题一的（3）的拓展

（一）模型的建立

即便我们已经很好地解决了问题一的（3）题，并对其收敛性、稳定性给出了很高的认可，但如果考虑其收敛速度和无人机行进总路程，可能效果并非最优。因此我们增强假设，将队列整体与各自的最优位置间距之和设为损失函数，提出了一种以损失函数最小化为调整依据的编队调整算法，结果证明该算法的总体无人机移动路径最小且误差减少更快。

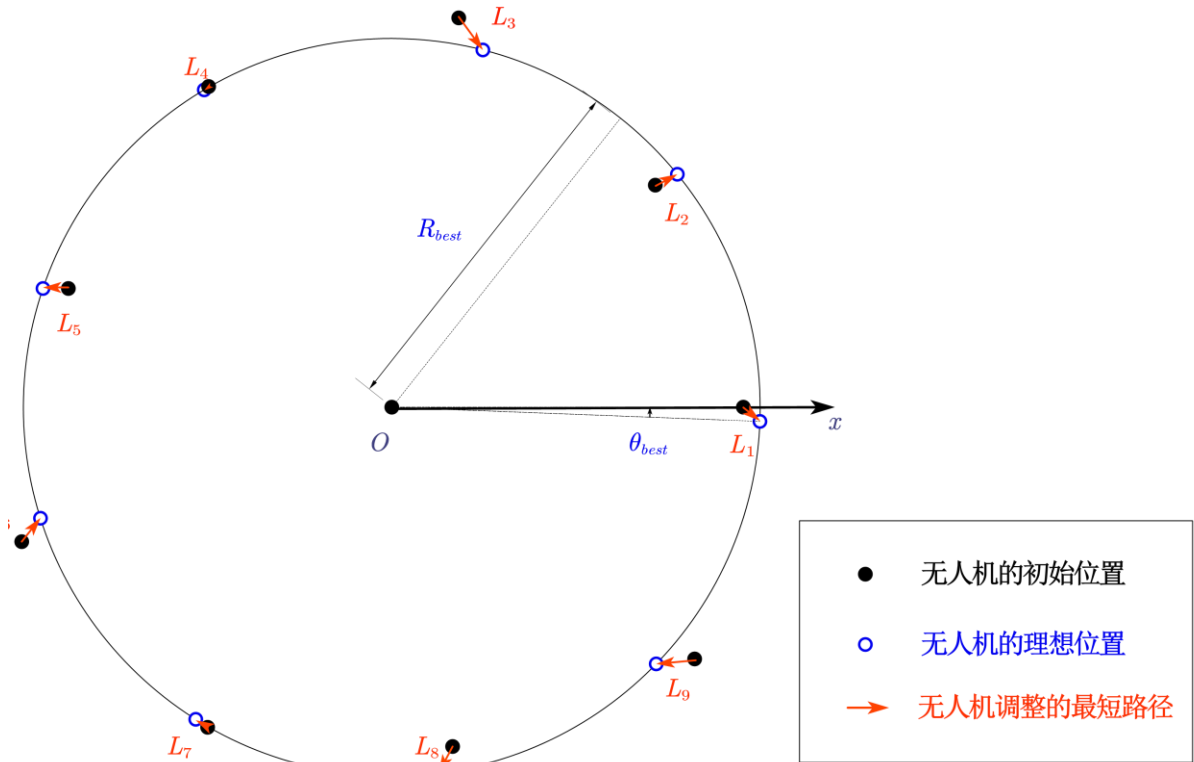
1. 建模最优圆周与最优坐标分布：由于九架无人机最终均匀分布在某个圆周

上，我们认为合理的无人机位置调整方案需要满足“所有无人机移动距离最小”的条件，为此我们首先要求解对应的最优圆周的半径与圆周上的九架无人机最优坐标。设九架无人机的最优坐标为 $\left(R_{best}, \theta_{best} + \frac{2\pi}{9}(k-1)\right), k = 1, 2, \dots, 9$ ，其中 R_{best} 和 θ_{best} 分别表示最优半径与最优起始角度。设圆周上九架无人机中第 k 架的初始坐标为 $(\rho_k, \theta_k), k = 1, 2, \dots, 9$ ，其中 ρ_k 和 θ_k 表示第 k 架无人机的极径和极角。根据表中的无人机初始位置，我们确定参数 $R_{best} \in [98m, 112m], \theta_{best} \in \left[-0.3 \frac{\pi}{180} rad, 0.3 \frac{\pi}{180} rad\right]$ 。为使所有无人机移动距离最小，我们只需设各点到达各自最优位置的距离之和最短即可，因此将目标函数设为

$$L = \sum_{k=1}^9 \sqrt{\left\{R_{best}^2 + \rho_k^2 - 2R_{best}\rho_k \cos\left[\theta_{best} + \frac{2\pi}{9}(k-1) - \theta_k\right]\right\}} \quad (6)$$

因此最优的半径与最优起始坐标为

$$R_{best}, \theta_{best} = \underset{R_{best}, \theta_{best}}{\operatorname{argmin}} \sum_{k=1}^9 \sqrt{\left\{R_{best}^2 + \rho_k^2 - 2R_{best}\rho_k \cos\left[\theta_{best} + \frac{2\pi}{9}(k-1) - \theta_k\right]\right\}} \quad (7)$$



图表 12 圆形无人机编队最优圆周和最优坐标示意图

2. 迭代更新:

a. 选择最优发射信号的无人机组合: 我们首先计算圆周上九架无人机与其对应的最优位置的距离, 将距离最小的两架无人机与圆心处的无人机作为最优的三架发射信号的无人机, 其余无人机不发射信号。

b. 估计不发射信号的无人机位置: 为了简化模型、降低运算复杂度, 我们将圆周上被选作发射信号的两架无人机位置近似看作处于其最优位置, 此时可以利用问题一(1)的模型对不发射信号的无人机位置进行定位。

c. 调整所有不发射信号的无人机位置: 设圆周上编号为 k 的无人机在第 i 轮迭代的坐标为 $(\rho_k^{(i)}, \theta_k^{(i)})$, $k = 1, 2, \dots, 9, i \in N$, 其对应的最优坐标为 $(R_{best}, \theta_{k_{best}})$, 则该无人机的移动矢量 $\vec{M}_k^{(i)}$ 为 $(R_{best} - \rho_k^{(i)}, \theta_{k_{best}} - \theta_k^{(i)})$ 。但考虑到圆周上发射信号的两架无人机位置做了近似处理, 且需保证“所有无人机移动距离最小”, 为此我们引入延迟因子 λ , 用于控制该无人机的行进距离, 在此我们给出 λ 的两种可行方案:

i. 令 λ 为属于 $(0,1]$ 的定值 λ_{const} 。

ii. 令 λ 随迭代次数指数衰减, 假设初值 $\lambda^{(1)} = \frac{1}{\lambda_{decrease}}$, 则令 $\lambda^{(i+1)} = \frac{1}{\lambda_{decrease}} \lambda^{(i)}$ 。

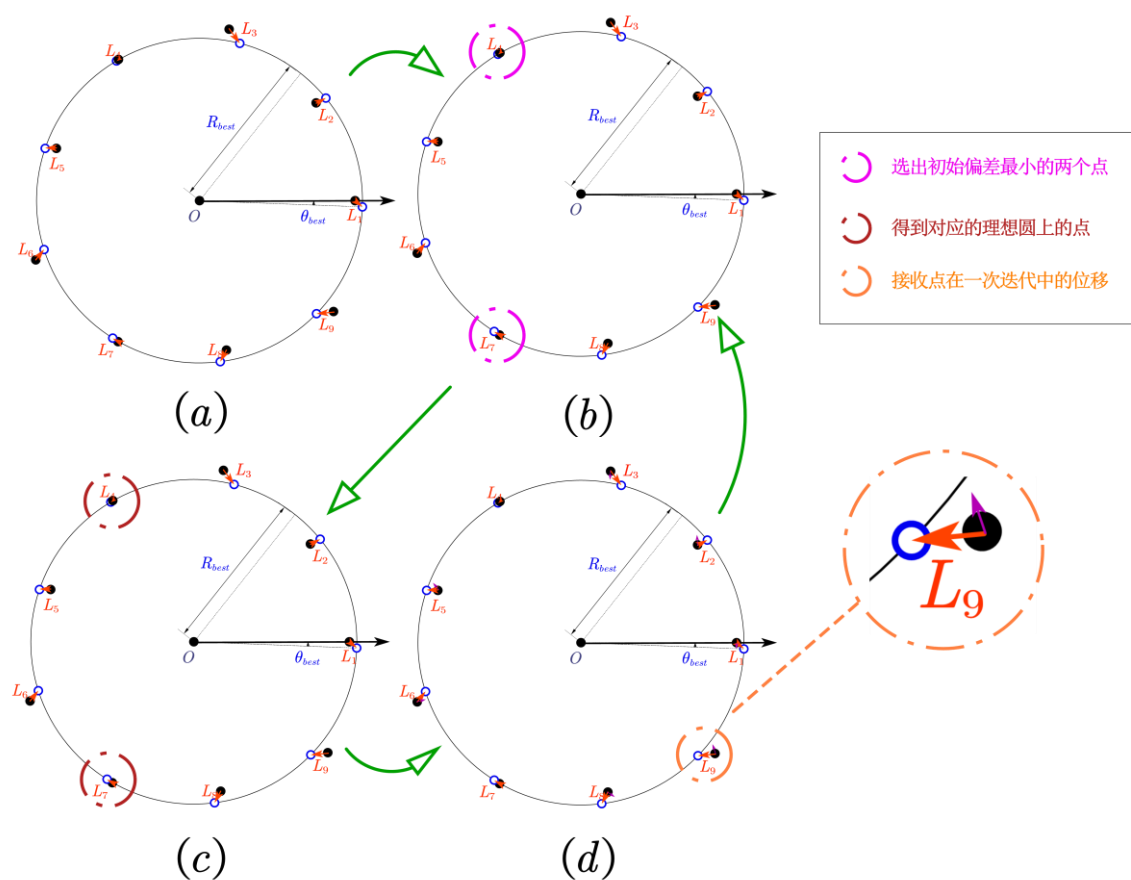
令该无人机的移动矢量 $\vec{M}_k^{(i)} = \lambda(R_{best} - \rho_k^{(i)}, \theta_{k_{best}} - \theta_k^{(i)})$ 。对所有不发射信号的无人机位置进行各自的移动矢量计算并移动位置, 将编号为 k 的无人机位置更新为

$$\begin{aligned} (\rho_k^{(i+1)}, \theta_k^{(i+1)}) &= ((1 - \lambda)\rho_k^{(i)} + \lambda R_{best}, (1 - \lambda)\theta_k^{(i)} + \lambda \theta_{k_{best}}) \\ k &= 1, 2, \dots, 9, i \in N \end{aligned} \quad (8)$$

d. 目标函数评估: 将圆周上的无人机坐标与其各自对应的最优坐标距离之和设置为目标函数 L' , 则

$$L'_{(i+1)} = \sum_{k=1}^9 \sqrt{R_{best}^2 + (\rho_k^{(i+1)})^2 - 2R_{best}\rho_k^{(i+1)} \cos\left[\theta_{k_{best}} + \frac{2\pi}{9}(k-1) - \theta_k^{(i+1)}\right]} \quad (9)$$

设置可接受的最大误差为 ϵ , 判断是否满足 $L' \leq \epsilon$, 若满足条件则停止迭代, 否则返回步骤 a 迭代。

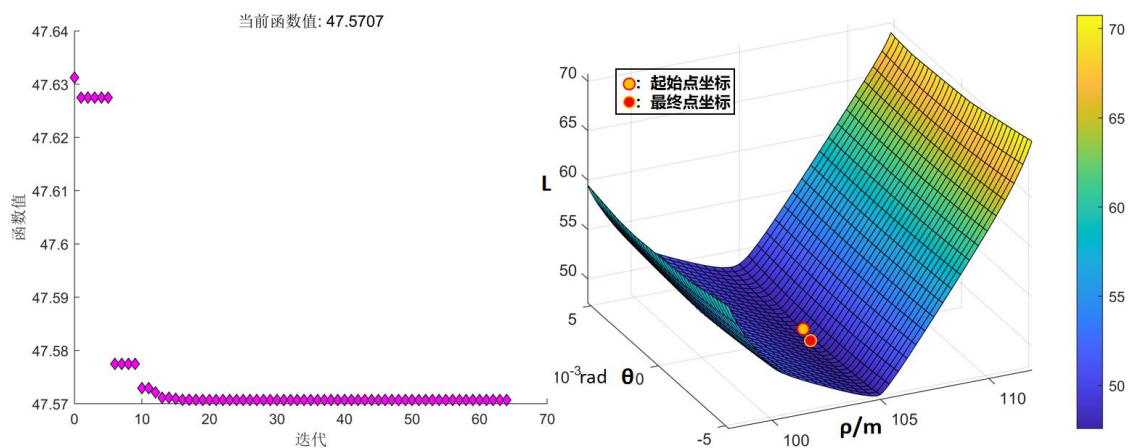


图表 13 以损失函数为调整依据迭代算法示意

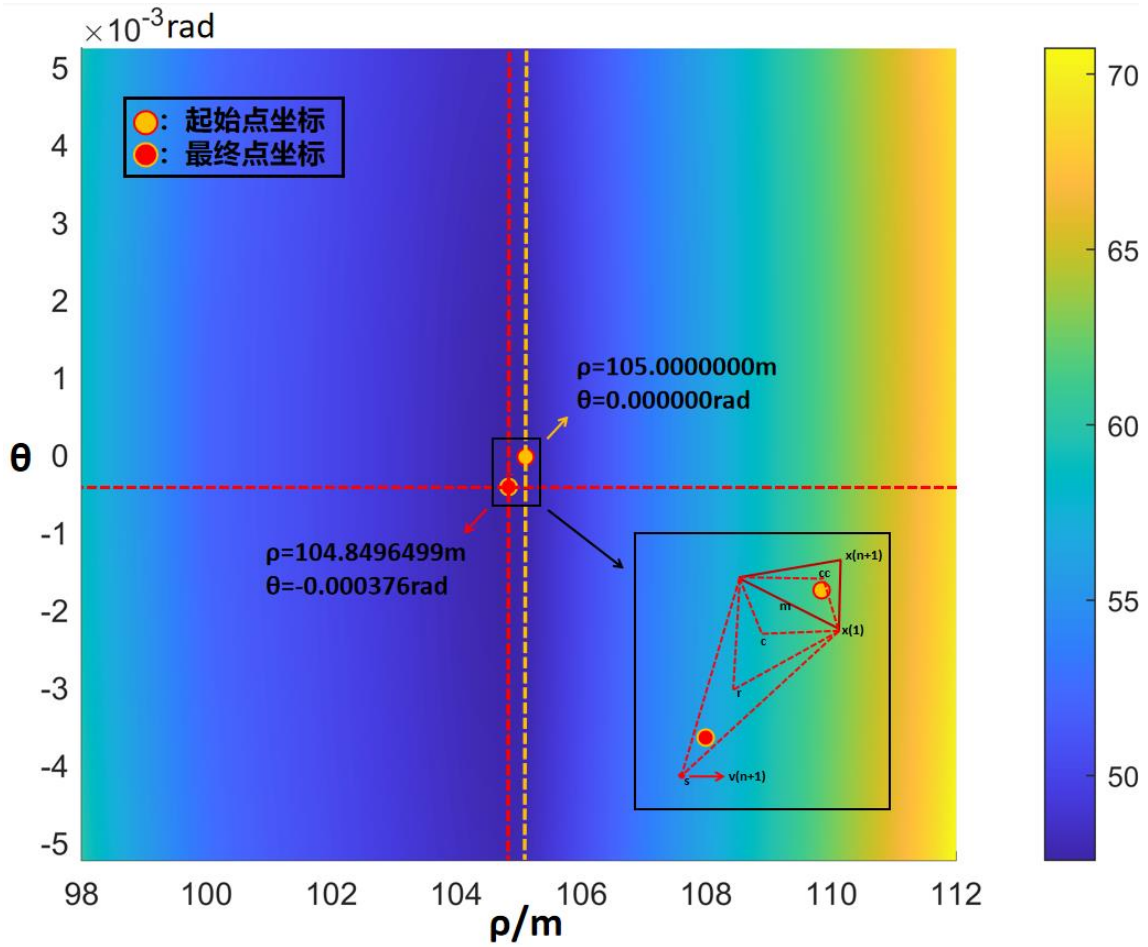
(二) 模型的求解

1. 求解最优圆周与最优坐标分布：使用 *Nelder – Mead* 单纯形算法[1]，算法示意图如图表 15，具体算法流程详见附录 4_算法 1。

解得： $R_{best} = 104.8496499m$, $\theta_{best} = -0.000376rad = -0.021569^\circ$



图表 14 目标函数迭代示意图



图表 15 Nelder-Mead 单纯形算法

2. 迭代实验：

我们设定迭代次数为200次，并将评价模型的指标定为：

- 在第200轮迭代时每架无人机与其最优位置的距离之和的均值。用该指标越小，代表无人机编队策略的调整精度越高。
- 在这200次迭代过程中的所有无人机移动距离之和。该指标越小，代表该策略需要无人机作更少的改动。
- 模型稳定的速度。假设第 N 次迭代时，误差已趋于稳定，则 N 值可以用于衡量该指标，且 N 越小，代表模型能够在更短的时间内完成编队的调整。但题干中说明不考虑时间因素，因此在本题中不做主要考虑，但考虑到实际意义，我们将 N 视为次要指标。

为便于后续表述，我们将上述三个指标依次简记为： V_{loss} 、 $V_{distance}$ 、 V_{speed}

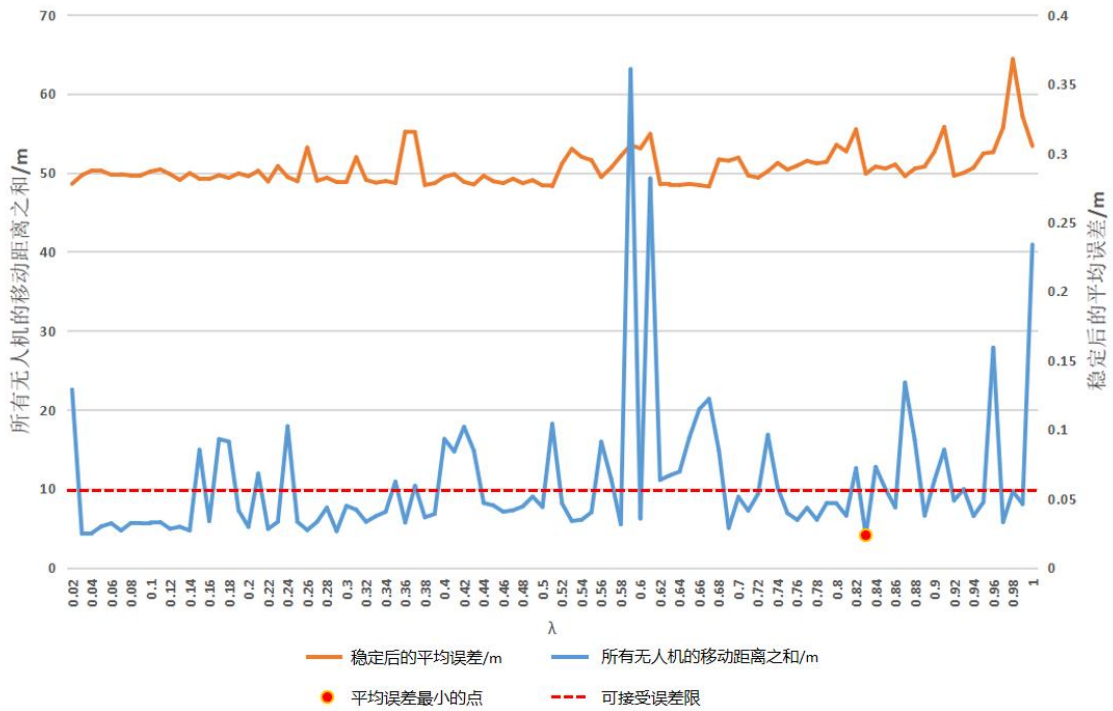
在不考虑延迟因子($\lambda = 1$)时，如图表 16 所示，平均误差随迭代次数增加

而趋于稳定。

之后我们根据对 λ 的两种设计方案分别进行消融实验，根据图图表 17 及附录 3_表格 2~3，我们分析模型性能并给出最优的 λ 选取方案：

方案一： λ 为定值 λ_{const} ，以 0.01 的步长从 0.01 到 1 遍历 λ_{const} 。

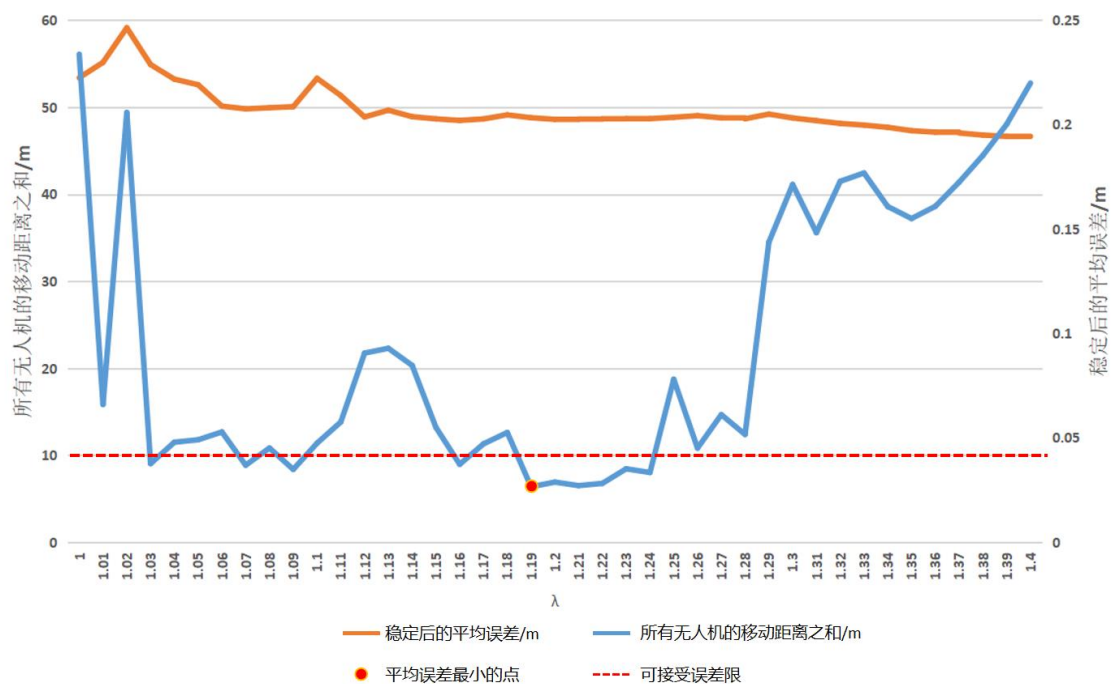
- 当迭代次数为 200 次时，对于不同的 λ ，平均误差均会降至某值并趋于稳定，其中当 $\lambda = 0.83$ 时， V_{loss} 最小。
- 假设每架无人机平均偏离最优位置小于 5cm 是可接受的误差，那么在所有可接受的误差点内，当 $\lambda = 0.5$ 时， $V_{distance}$ 最小。



图表 16 方案一中不同 λ 对模型性能的影响

方案二： λ 随迭代次数衰减，每次迭代为之前的 $\frac{1}{\lambda_{decrease}}$ ，以 0.01 的步长从 1.00 到 1.40 遍历 $\lambda_{decrease}$ 。

- 在迭代次数为 200 次时，对于不同的 λ ，平均误差均会降至某值并趋于稳定，其中当 $\lambda = 1.19$ 时， V_{loss} 最小。
- 假设每架无人机平均偏离最优位置小于 5cm 是可接受的误差，那么在所有可接受的误差点内，当 $\lambda = 1.20$ 时， $V_{distance}$ 最小。



图表 17 方案二中不同 λ 对模型性能的影响

表格 2 不同方案下的最优 λ 的性能指标

	$\lambda = 0.50$	$\lambda = 0.83$	$\lambda = 1.00$	$\lambda = 1.19$	$\lambda = 1.20$
V_{loss}	0.0438382	0.023471	0.233722	0.0265913	0.0288387
$V_{distance}$	48.3907	49.8953	53.3887	48.7947	48.6113
V_{speed}	17	8	4	12	15

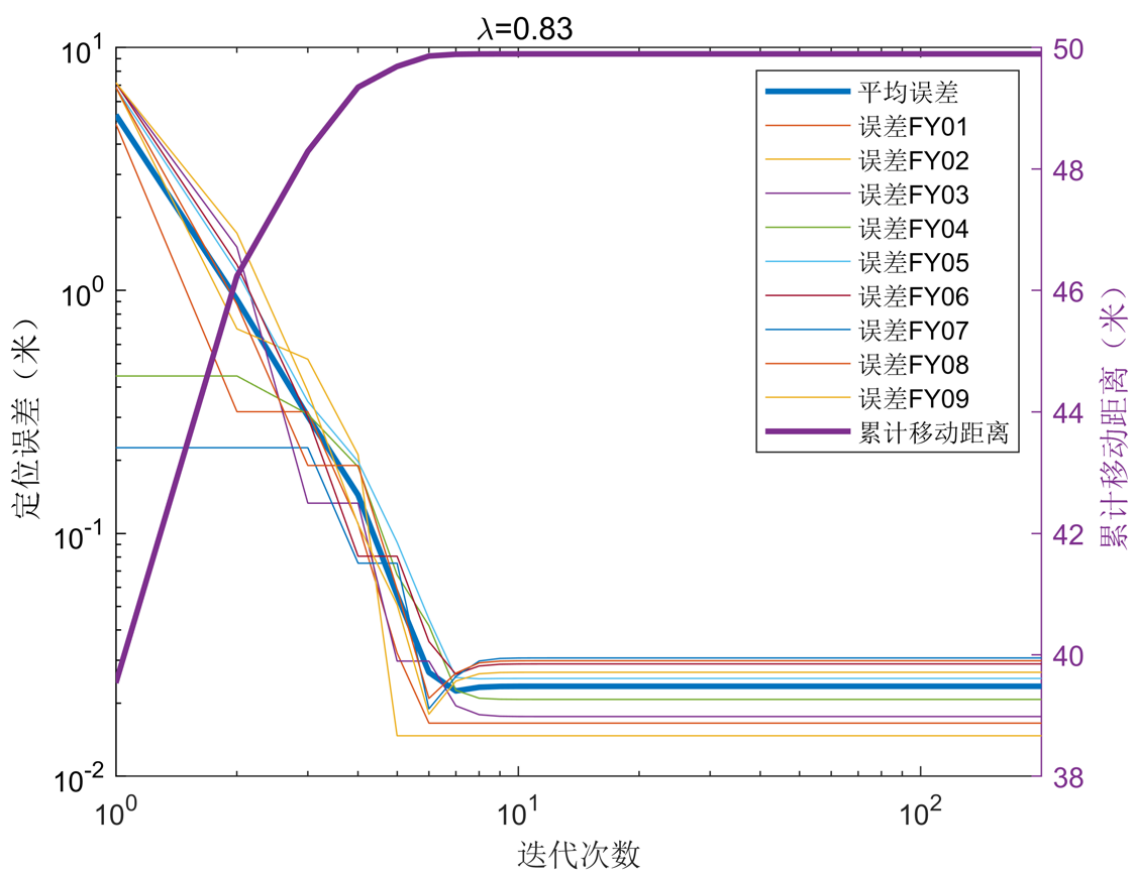
限于篇幅，仅展示 $\lambda = 0.83$ 的数据曲线如图表 18，其余四种情况见附录 2_图表 1~4。

因此可根据实际需求进行 λ 的选取：

- 追求误差小：选择 $\lambda = 0.83$ 。
- 追求更少的移动总距离：选择 $\lambda = 0.50$ 。
- 追求更快的稳定速度：选择 $\lambda = 1.00$ 。

此外，我们的无人机编队调整算法是一种动态调整策略，即随迭代次数的增加，所有无人机均在各自的最优位置附近小幅度动态调整。从理论上而言，当无人机编队在受外力扰动时，只要扰动小于初始扰动，模型总能够迅速实现

动态调整，且最终误差在可接受范围内。



图表 18 当 $\lambda = 0.83$ 时的模型效果图

5.2 问题二模型的建立与求解

由于问题二给定的条件很少，所以我们做出假设如下：

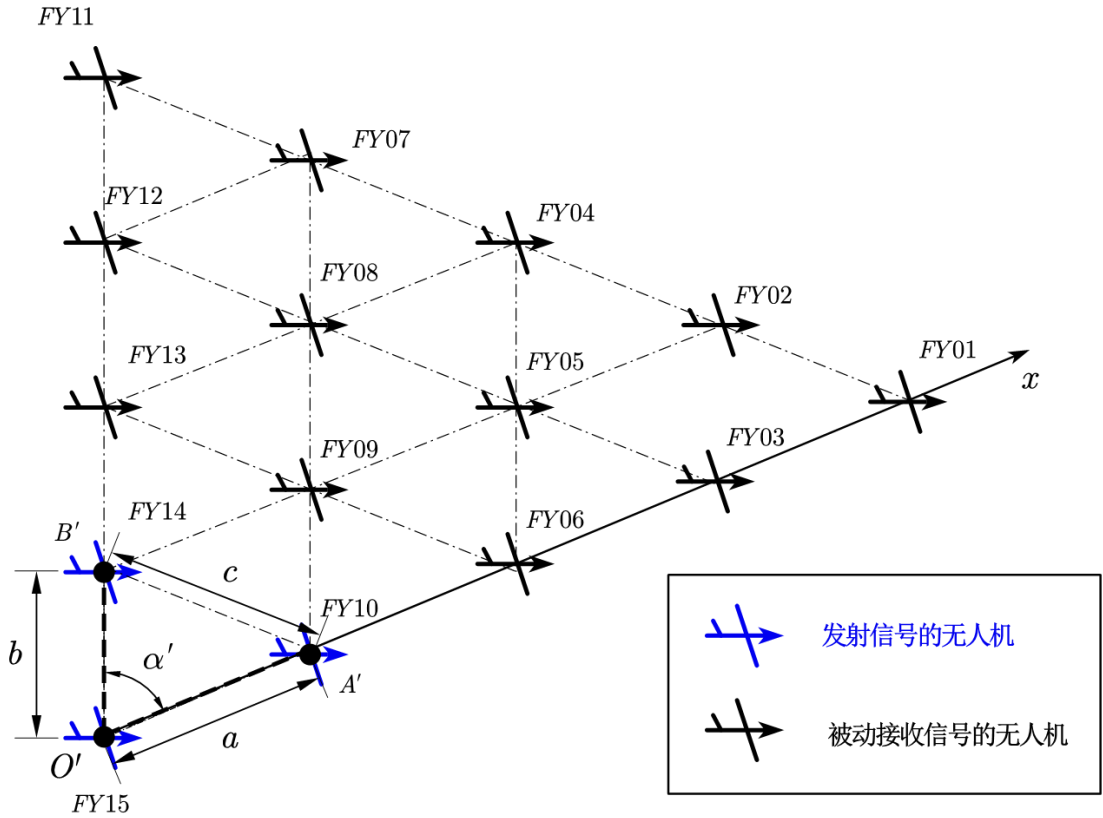
1. 无人机基于自身感知的高度信息，均保持在同一个高度上飞行。
2. 假定最小的三角形单元的三边已知。
3. 锥形编队队形中位于角上的某一架飞机（FY01，FY11或FY15）作为基准飞机，再配合任意两架无人机发射信号，其余位置略有偏差的无人机被动接收信号。
4. 发射信号的无人机位置无偏差且编号已知。
5. 锥形编队调整位置前后，各个无人机的相对位置不发生改变。

5.2.1 定位模型

（一）模型的建立

1. 为表述清晰，以FY10作为第二架发射信号的无人机进行建系。
2. 建立极坐标系：以FY15为坐标原点 O' ，由FY15指向FY10的射线作为极轴 $O'x$ 建立极坐标系，沿用问题一中极坐标的拓展定义。设最小的三角形单元的三边分别为 a, b, c ，规定角度取逆时针方向为正，以 O' 为顶点且由 A' 逆时针指向 B' 的角度为

α' ，做出示意图如图表 199 所示。

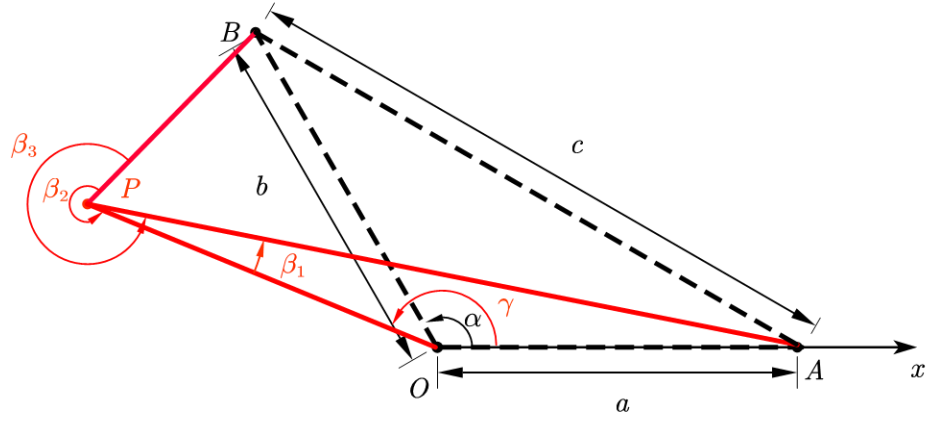


图表 19 锥形无人机编队的极坐标系

3. 根据三角形全等得到锥形编队队形的边长：因为锥形编队队形直线上相邻两架无人机间距相等，由平面几何中三角形全等的知识，可得最小的三角形单元相互全等，编队构成的最大三角形边长分别为 $4a, 4b, 4c$ 。

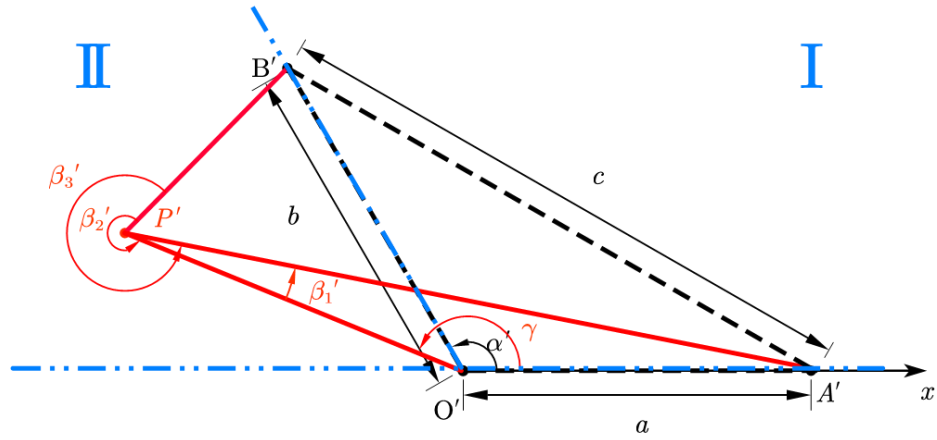
4. 构建扰动模型：沿用圆形编队中的设定，将微小扰动导致被动定位的无人机的位置变化分解为径向变化 δ_{θ}' 和角度扰动 δ_R' ，规定变化范围为 $\delta_R' \in [-0.07l, 0.07l]$ ，其中 l 为被动定位的无人机到极点的距离， $\delta_{\theta}' \in [-0.30^\circ, 0.30^\circ]$ ，且扰动在范围内均匀分布。

5. 建立纯方位无源定位模型（发射信号的无人机共三架且编号已知）：设被动接收信号的无人机位置为 P' 。为了使模型具有普适性，同时可以统一 β' 角间的代数关系，我们沿用问题一的（1）中的规定，此时有 $\beta_3' = \beta_1' + \beta_2'$ 成立。设测量出接收角 $\angle O'P'A' = \alpha_1, \angle B'P'O' = \alpha_2, \angle A'P'B' = \alpha_3$ 。建立模型如图表 20 所示。



图表 20 锥形无人机编队中的定位模型

3. 构建接收角与方向角的对应关系：该模型中的接收角与计算使用的有向角同样存在不相等的情况。值得强调的是，因为我们选定角上的无人机作为基准的发射信号无人机，所以 γ' 一定小于等于该无人机所在角的大小，我们不妨设该角为 ϕ ，而三角形的内角一定是劣角，即 $\phi < 180^\circ$ ，所以有不等式： $\gamma' \leq \phi < 180^\circ$ ，故只需要划分为两个区域I和II即可，区分划分仍以 $O'A'$ 和 $O'B'$ 所在直线为分割线，如图表 21 所示



图表 21 分区域的锥形无人机编队中的定位模型

在不同区域中，接收角（劣角）和有向角的对应关系如下：

表格 3 锥形无人机编队接收角（劣角）和有向角的对应关系

四个区域	对应关系
第一区域 (I)	$\beta_1' = \alpha_1', \beta_2' = \alpha_2', \beta_3' = \beta_1' + \beta_2' = \alpha_1' + \alpha_2'$
第二区域 (II)	$\beta_1' = \alpha_1', \beta_2' = 2\pi - \alpha_2', \beta_3' = \beta_1' + \beta_2' = 2\pi + \alpha_1' - \alpha_2'$

（二）模型的求解

1. 接收角转化为有向角：与问题一的（1）思路一致，在得到接收角 $\alpha'_1, \alpha'_2, \alpha'_3$ 后，根据已知编号，利用被动定位的无人机与发射信号的无人机的相对位置，确定被动定位的无人机的位置所在区域，通过表格 3 中接收角和有向角的对应关系计算出有向角 $\beta'_1, \beta'_2, \beta'_3$ 。

2. 利用余弦定理求解未知角 α' ：在 $\Delta B'O'A'$ 中由余弦定理有： $c^2 = a^2 + b^2 - 2ab \cos \alpha'$ ，即 $\alpha' = \arccos \frac{a^2 + b^2 - c^2}{2ab}$ 。

3. 计算思路与问题一的（1）一致，在 $\Delta O'A'P'$ 和 $\Delta O'B'P'$ 中通过公共边 $O'P'$ 分别使用正弦定理构造二元方程求解未知量 $O'P'$ 和 γ' ，以此来得到被动接收信号的无人机的极坐标位置。

a. 在 $\Delta O'A'P'$ 中有：

$$\frac{\sin \beta'}{a} = \frac{\sin(\gamma' + \beta'_1)}{O'P'} \quad (10)$$

b. 在 $\Delta O'B'P'$ 中有：

$$\frac{\sin \beta'_2}{b} = \frac{\sin(\beta'_2 + \alpha' - \gamma')}{O'P'} \quad (11)$$

c. 两式相除可得：

$$\frac{a \sin \beta'_2}{b \sin \beta'_1} = \frac{\sin(\alpha' + \beta'_2 - \gamma')}{\sin(\gamma' + \beta'_1)} \quad (12)$$

d. 化简为：

$$\tan \gamma' = \frac{b \sin \beta'_1 \sin(\beta'_2 + \alpha') - a \sin \beta'_1 \sin \beta'_2}{b \sin \beta'_1 \cos(\beta'_2 + \alpha') + a \sin \beta'_2 \cos \beta'_1} \quad (13)$$

同样，因为对极坐标进行了拓展定义， (ρ, θ) ， $(-\rho, \theta + \pi + 2nk)$ ，表示的是同一个位置，其中 $n \in \mathbb{Z}$ ，所以不需要对 γ' 的范围进行讨论与问题。但不同的是因为锥形编队队形中最小三角形单元的边长 (a, b, c) 都未知，所以 γ' 可能为 $\frac{\pi}{2} + n\pi, n \in \mathbb{Z}$ ，需要考虑 $\tan \gamma'$ 不存在的情况，故分类讨论，将 γ' 表示为：

$$\gamma = \begin{cases} \frac{\pi}{2}, & b \sin \beta'_1 \cos(\beta'_2 + \alpha') + a \sin \beta'_2 \cos \beta'_1 = 0 \\ \arctan \left[\frac{b \sin \beta'_1 \sin(\beta'_2 + \alpha') - a \sin \beta'_1 \sin \beta'_2}{b \sin \beta'_1 \cos(\beta'_2 + \alpha') + a \sin \beta'_2 \cos \beta'_1} \right], & b \sin \beta'_1 \cos(\beta'_2 + \alpha') + a \sin \beta'_2 \cos \beta'_1 \neq 0 \end{cases} \quad (14)$$

其中， $\alpha' = \arccos \frac{a^2 + b^2 - c^2}{2ab}$ 。

计算无人机 P' 在极坐标系内的极径 $O'P'$ ：可得 $O'P' = \frac{\sin(\gamma' + \beta'_1)}{\sin \beta'_1} a$ ，即解得 P' 在

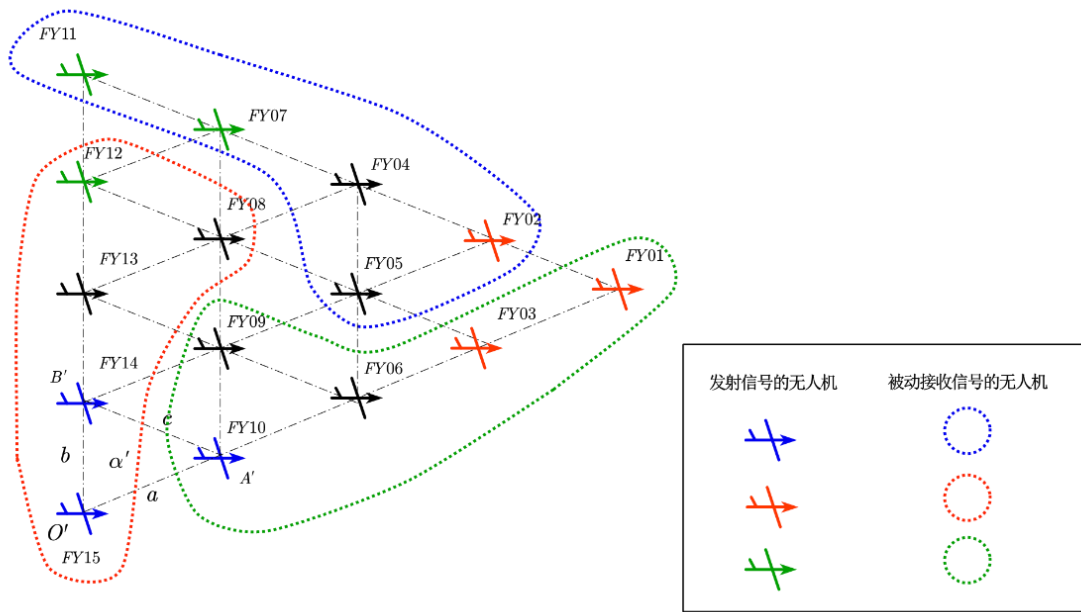
极坐标下的位置为 $P\left(\frac{\sin(\gamma' + \beta'_1)}{\sin \beta'_1} a, \gamma'\right)$ 。

5.2.1 编队模型

(一) 模型的建立

1. 假设以锥形无人机编队的三个角上的最小三角形单元队列为发射信号的无人机，则共有三组无人机发射信号，为了实现整个锥形无人机队列的位置调整，接收信号的无人机也应为三组，故取极限情况下每五个无人机为一组作为接收信号的无人机。
2. 因为每架无人机都存在扰动，在无人机扰动的位移一定的情况下，发射信号的无人机与被动接收信号的无人机距离越远，扰动导致的相对位置变化越小，故综合考虑三组发射信号的无人机与被动接受信号的无人机对应的距离关系，设每组内各发射-接收无人机对的距离之和为 $l_k, k=1,2,3$ ，以

$\max \sum_{k=1}^3 l_k, k=1,2,3$ 为目标函数，计算可得最佳分组如图表 22 所示。



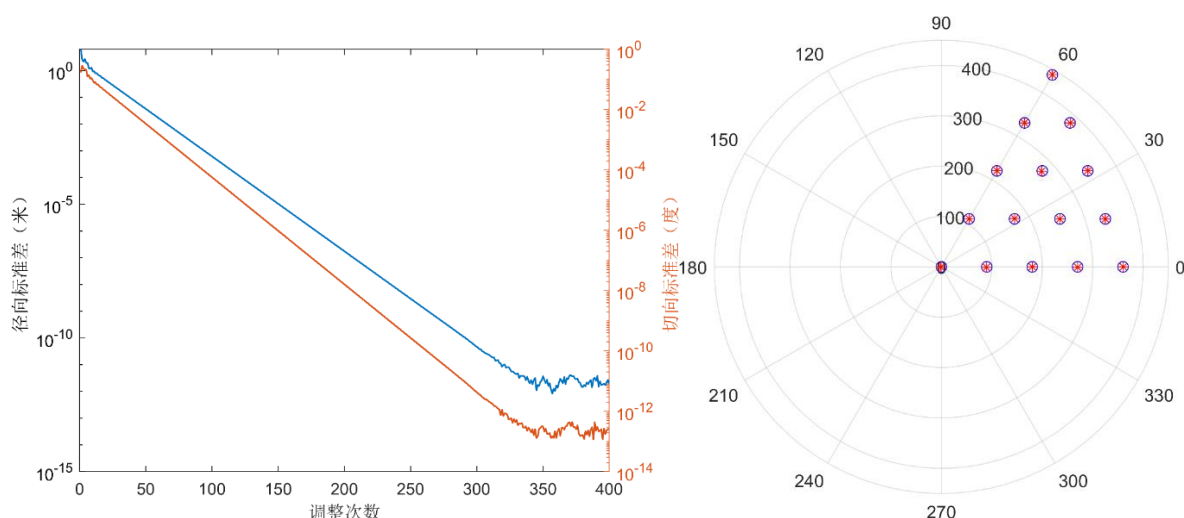
图表 22 锥形编队队形调整的最优的分组方案

3. 以每架发射信号的无人机为中心点，被动接受信号的无人机到中心点的距离为径向长度。因为锥形无人机编队的最小三角单元 a, b, c 已知，根据三角形全等知识，队形中不同编号的无人机位置信息已知，至此，问题已经转化为径向移位迭代模型，带入计算即可。

(二) 模型的验证：

按照迭代过程执行 400 次迭代，迭代结果如图表 23 所示。可见无人机的径向距离和切向距离误差均在 400 次迭代后达到机器精度 (10^{-11})，证明了该算法在圆形

无人机编队和锥形无人机编队中的可迁移性。



图表 23 锥形无人机编队的径切向移位迭代模型的迭代结果示意图（左）

图表 24 调整前后位置对比图（右）

六、模型的评价、改进与推广

6.1 模型的优缺点

1. 问题一（1）的“三架无人机编队纯方位定位模型”：
 - a) 优点：利用正弦定理等数学原理成功推导出在极坐标系下的未知点坐标公式，因此其算法复杂度极低，且误差几乎为 0。
2. 问题一（2）的“基于扰动模型和蒙特卡洛的多分类模型”：
 - a) 优点：该模型利用蒙特卡洛模拟，在允许范围内的扰动下能够在“接收角域”内进行有效区分，且不交叠，能够将问题成功转为问题一（1）的模型。
 - b) 缺点：若扰动过大会导致“接收角域”交叠，无法通过多分类算法实现未知发射机的有效定位。
3. 问题一（3）的“径切向移位迭代模型”：
 - a) 优点：充分利用圆形的拓扑结构和几何性质，较根据损失函数调整编队的补充算法而言，具有如下优点：
 - i. 算法是收敛的。
 - ii. 算法收敛至平衡位置的速度更快。
 - iii. 由于算法收敛，模型具有更强的稳定性。
 - b) 缺点：编队各无人机行进的绝对路径总和更长。
4. 问题二的“锥形无人机编队纯方位无源定位/编队模型”：
 - a) 优点：充分发挥了模型的可迁移性，一方面证明了“径切向移位迭代模型”的可拓展性，另一方面减少了在不同无人机编队队形调整中的工作量。

6.2 模型的改进

问题一（2）中的蒙特卡洛模拟仅得到了在假设的扰动范围下，第三架发射飞机的编号可以分辨的结果。可以考虑求出可分辨的扰动范围，建立普适性更强的模型。

6.3 模型的推广

1. 我们已经验证了三架无人机编队纯方位定位模型在圆形编队队形和锥形编队队形中的可迁移性，可以考虑该定位模型在其他编队队形下是否仍可沿用，进而总结出在某种拓扑结构下的普适模型。

2. 我们分析了不同条件下，径切向移位迭代调整模型在圆形编队队形和锥形编队队形中的迁移性，可以考虑该调整模型在其他情况下的适用性，尤其是其他在几何性质上与圆和或锥形类似的模型，尝试总结出普适化模型。

七、参考文献

- [1]. Lagarias, J. C., J. A. Reeds, M. H. Wright, and P. E. Wright. “Convergence Properties of the Nelder-Mead Simplex Method in Low Dimensions.” *SIAM Journal of Optimization*, Vol. 9, Number 1, 1998, pp. 112–147.

支撑文件目录

文件名列表

表格

- 表格 1: 问题一（1）的结果数据.xlsx
- 表格 2: 问题一的（3）lambda_decrease.csv
- 表格 3: 问题一的（3）lambda_const.csv

代码

- classify_check02.m
 - move03.m
 - pre_calc03.m
 - question01.m
 - simple_21.m
 - simple03.m
-

附录

附录 1_代码

问题一 (1)

```
%% 问题一(1)
%% 代入理论计算，确认确实可以成功计算得无人机自身位置（1e-13 量级误差）
% 并展示部分验证结果

clc, clear, close all, format short g
% 预处理标准位置
R=100;
std_rho=[0; ones(9,1)*R];
std_theta=[0; (0:8)'/9*2*pi];
global now_pos std_pos
std_pos=std_rho.*[cos(std_theta) sin(std_theta)];

color=["#0072BD"
       "#D95319"
       "#EDB120"
       "#7E2F8E"
       "#77AC30"
       "#4DBEEE"
       "#A2142F"];
tic
% 记录全部单次误差结果
XX=[];
YY=[];
for third=2:5
    figure(2)
    subplot(220+third-1)
    hold on
    % 绘制发射机
    id=[0 1 third]+1;
    scatter(std_pos(id,1),std_pos(id,2),'filled');
    % 绘制目标圆周
    ax=rectangle('Position',[-R,-R,2*R,2*R],'Curvature',[1,1]);
    ax.LineStyle='--';
    axis equal
```

```

    fprintf('随机生成的第三架发射信号的无人机编号:
FY%02d\n',third);
    X=[];
    Y=[];
    for self=2:9
        if self==third
            continue
        end
        fprintf('随机生成的被动接收信号的无人机编号:
FY%02d\n',self);
        % 随机自身位置并判断自身位置
        % 径向偏移区间为[-0.02, 0.12]*R
        error_rho=14*rand-2;
        % 角度偏移区间为[-0.3°, 0.3°]
        error_theta=0.3*pi/180*(2*rand-1);
        % 更新自身偏移
        now_pos=std_pos;
        self_rho=std_rho(self+1)+error_rho;
        self_theta=std_theta(self+1)+error_theta;
        now_pos(self+1,:)=self_rho.*[cos(self_theta)
sin(self_theta)];
        scatter(now_pos(self+1,1),now_pos(self+1,2),'bo')
        % 由于无人机知道发射信号的无人机编号和其相对方向，因此也很容易将角度和无人机编号对应
        angle=[getangle(self,0,1) getangle(self,0,third)
getangle(self,1,third)];
        % ∠OPA ∠BPO ∠BPA
        dir=([getstdir(self,0,1) getstdir(self,third,0)
getstdir(self,third,1)]>0)*2-1;
        beta=angle.*dir;
        if abs(beta(1)+beta(2)-beta(3))>1e-9
            error('有向角错误')
        end
        alpha=40*(double(third)-1)*pi/180;
        gama=atan((sin(beta(1)) * (sin(alpha + beta(2)) -
sin(beta(2)))) / (sin(beta(1)) * cos(alpha + beta(2)) +
cos(beta(1)) * sin(beta(2))));
        my_error=sin(beta(1))*sin(alpha+beta(2)-gama)-
sin(beta(2))*sin(gama+beta(1));

```

```

    if my_error>1e-9
        error('方程错误')
    end
    OP=R*sin(gama+beta(1))/sin(beta(1));
    calc_pos=OP*[cos(gama) sin(gama)];
    scatter(calc_pos(1),calc_pos(2),'r*')
    % now_pos(self+1,:)
    my_error=norm(calc_pos-now_pos(self+1,:));
    if my_error>1e-9
        error('计算位置 and 实际位置偏差过大 (%gm)',my_error)
    end
    X=[X; sprintf('FY%02d',self)];
    Y=[Y my_error];
end
if third==3
    legend('发射机','接收机','计算位置','Position',[0.9 0.6
0.05 0.05])
end
figure(3)
subplot(220+third-1)
hold on
tmp=mat2cell(X,ones(7,1));
X = categorical(tmp);
X = reordercats(X,tmp);
bar(X,Y)
XX=[XX X];
YY=[YY Y'];
ylabel('误差/米')
end
figure(2)
suptitle('误差（平面图）')
figure(3)
suptitle('误差（条形图）')
toc

```

```

function ret=getstdir(self,x,y)
    % 使正弦定理计算标准角度方向
    global std_pos
    a=std_pos(x+1,:)-std_pos(self+1,:);
    b=std_pos(y+1,:)-std_pos(self+1,:);
    ret=a(1)*b(2)-a(2)*b(1);
end

function ret=getangle(self,x,y)
    % 使余弦定理计算应该测量得到的角度值
    global now_pos
    a=norm(now_pos(x+1,:)-now_pos(self+1,:));
    b=norm(now_pos(y+1,:)-now_pos(self+1,:));
    c=norm(now_pos(x+1,:)-now_pos(y+1,:));
    ret=acos((a^2+b^2-c^2)/(2*a*b));
end

```

问题一（2）

```

%% 问题一(2)
%% 蒙特卡洛模拟
% 由对称性, 不妨令接收信号的无人机取 FY02 ~ FY05
%% 回代计算接收到的第三架老人机的编号 && 计算自身的位置 并验证
% 接收信号的无人机与发射信号的无人机任取 FY02 ~ FY09, 同时验证
蒙特卡洛模拟时使用对称性的合理性
clc, clear, close all
% 预处理标准位置
R=100;
std_rho=[0; ones(9,1)*R];
std_theta=[0; (0:8)'/9*2*pi];
global now_pos std_pos
std_pos=std_rho.*[cos(std_theta) sin(std_theta)];
now_pos=std_pos;
% 预设图窗大小
set(gcf, 'Position', [0 0 800 800]);

color=["#0072BD"
       "#D95319"
       "#EDB120"
       "#7E2F8E"
       "#77AC30"
       "#4DBEEE"
       "#A2142F"];

tic
% 重复次数, 5000 次用于调试, 约 2 秒出结果; 50000 次提高精度
ts=500;
% 预分配内存以获得更高的运算速度
angles=zeros(ts,2);
% 分别计算四种情况 (另外四种可以由对称性类似求解)
data=zeros(4,ts*7,2);
for self=2:5
    subplot(220+self-1)
    title(['self = ' num2str(self) ' (' num2str(11-self) ' is
similar)'])
    xlabel('smallest degree(°)')
    ylabel('second smallest degree(°)')
    axis([0 90 0 90])
    hold on
    l = [];
    % 枚举选取的第三架无人机

```

```

for third=2:9
% 选择的第三架无人机不能为自己
if self==third
continue
end
% 蒙特卡洛模拟，反复随机自身的位置
for test=1:ts
% 径向偏移区间为 $[-0.02, 0.12] \times R$ 
error_rho=14*rand-2;
% 角度偏移区间为 $[-0.3^\circ, 0.3^\circ]$ 
error_theta=0.3*pi/180*(2*rand-1);
% 更新自身偏移
now_pos=std_pos;
self_rho=std_rho(self+1)+error_rho;
self_theta=std_theta(self+1)+error_theta;
now_pos(self+1,:)=self_rho.*[cos(self_theta)
sin(self_theta)];
% 获取自身测量所得的三个角度值
angle=[getangle(self,0,1) getangle(self,0,third)
getangle(self,1,third)];
angle=sort(angle);
% 检测两个小角和等于大角
if abs(angle(1)+angle(2)-angle(3))>1e-9
error('有向角错误')
end
% 保留两个小角
angles(test,:)=angle(1:2);
end
angles=angles*180/pi;
% 选择的第三架无人机不能为自己，重新标号
id=third-1-boolean(third>self);
data(self-1,(id-1)*ts+1:id*ts,:)=angles;
% 绘制自身位置的不同偏移情况时，所测量的角度的两个较小值的分布情况
plot(angles(:,1),angles(:,2),'.','MarkerSize',1,'Color',
color(id))
% 设置图例，并放大点以便于观察颜色
l = [1
plot(NaN,NaN,'.','MarkerSize',10,'Color',color(id),'DisplayNam
e',sprintf("FY%02d",third))];
end

```

```

        legend(1)
    end
    toc
    % 保存为 pdf 矢量图
    if ts >= 50000
        exportgraphics(gcf,'pic_hold.pdf','ContentType','vector')
        exportgraphics(gcf,'pic_hold.png','Resolution',600)
    end
    % 随机自身位置并判断接收到的信号来自哪架飞机
    tic
    figure, hold on
    ax=rectangle('Position',[-R,-R,2*R,2*R],'Curvature',[1,1])
    ax.LineStyle='--'
    axis equal
    scatter([R 0],[0 0])
    map=[0 1 2 3 4 4 3 2 1];
    for test=1:100
        % self=randi([2 9]);
        self=mod(test-1,8)+2;
        % 第三架飞机不能和自己相同
        third=self;
        while third==self
            third=randi([2 9]);
        end
        fprintf('随机生成的被动接收信号的无人机编号: FY%02d\n',self);
        fprintf('随机生成的第三架发射信号的无人机编号:
FY%02d\n',third);
        % 获得角度信息
        % 径向偏移区间为[-2, 12]
        error_rho=14*rand-2;
        % 角度偏移区间为[-0.3°, 0.3°]
        error_theta=0.3*pi/180*(2*rand-1);
        % 更新自身偏移
        now_pos=std_pos;
        self_rho=std_rho(self+1)+error_rho;
        self_theta=std_theta(self+1)+error_theta;
        now_pos(self+1,:)=self_rho.*[cos(self_theta)
sin(self_theta)];
        scatter(now_pos(self+1,1),now_pos(self+1,2),'bo')
    end
end

```

```

        % 获取自身测量所得的三个角度值
        angle=[getangle(self,0,1) getangle(self,0,third)
getangle(self,1,third)];
        angle=sort(angle);
        % 计算与最可能的飞机(最小二乘  $\theta_1^2+\theta_2^2$  最小)
        angle=angle(1:2)*180/pi;
        fprintf('其中较小的两个测量角为%f°, %f°\n',angle(1),
angle(2))
        tmp=squeeze(data(map(self),:,:));
        res=sum((tmp-angle).^2,2);
        [minval, minid]=min(res);
        if self<=5
            guess_third=idivide(int32(minid),ts,'ceil')+1;
            % 第三架飞机不能和自己相同, 从标号反求飞机序号
            if guess_third>=self
                guess_third=guess_third+1;
            end
        else
            guess_third=10-idivide(int32(minid),ts,'ceil');
            if guess_third<=self
                guess_third=guess_third-1;
            end
        end
        fprintf('计算得到的第三架无人机编号: FY%02d\n\n',
guess_third);
        if guess_third~=third
            error('计算得到的第三架无人机的编号不符合实际')
        end
        % 此时, 由于无人机知道发射信号的无人机编号和其相对方向, 因此也很容易将角度和无人机编号对应
        angle=[getangle(self,0,1) getangle(self,0,guess_third)
getangle(self,1,guess_third)];
        % ∠OPA ∠BPO ∠BPA
        dir=([getstdir(self,0,1) getstdir(self,guess_third,0)
getstdir(self,guess_third,1)]>0)*2-1;
        beta=angle.*dir;
        if abs(beta(1)+beta(2)-beta(3))>1e-9
            error('有向角错误')
        end
        alpha=40*(double(guess_third)-1)*pi/180;

```

```

        gama=atan((sin(beta(1)) * (sin(alpha + beta(2)) -
sin(beta(2)))) / (sin(beta(1)) * cos(alpha + beta(2)) +
cos(beta(1)) * sin(beta(2))));
        my_error=sin(beta(1))*sin(alpha+beta(2)-gama)-
sin(beta(2))*sin(gama+beta(1))
        if my_error>1e-9
            error('方程错误')
        end
        OP=R*sin(gama+beta(1))/sin(beta(1));
        calc_pos=OP*[cos(gama) sin(gama)]
        scatter(calc_pos(1),calc_pos(2),'r*')
        now_pos(self+1,:);
        my_error=norm(calc_pos-now_pos(self+1,:))
        if my_error>1e-9
            error('计算位置和实际位置偏差过大 (%gm) ',my_error)
        end
    end
end
toc

function ret=getstdidir(self,x,y)
    % 使正弦定理计算标准角度方向
    global std_pos
    a=std_pos(x+1,:)-std_pos(self+1,:);
    b=std_pos(y+1,:)-std_pos(self+1,:);
    ret=a(1)*b(2)-a(2)*b(1);
end

function ret=getangle(self,x,y)
    % 使余弦定理计算应该测量得到的角度值
    global now_pos
    a=norm(now_pos(x+1,:)-now_pos(self+1,:));
    b=norm(now_pos(y+1,:)-now_pos(self+1,:));
    c=norm(now_pos(x+1,:)-now_pos(y+1,:));
    ret=acos((a^2+b^2-c^2)/(2*a*b));
end

```

问题一（3）-1

```

%% 问题一(3)
%% 预求解直线移动最小的移动目标位置

clc, clear, close all, format long g
rho_list=[100,98,112,105,98,112,105,98,112]';
theta_list=[0,40.10,80.21,119.75,159.86,199.96,240.07,280.17,320.28]'/180*pi;
% 绘制无人机分布图并导出到文件
polarscatter(theta_list,rho_list)
exportgraphics(gcf,'compare-105.png','Resolution',300)
% 生成当前坐标表
now=rho_list.*[cos(theta_list) sin(theta_list)];
% 目标位置函数
aim=@(R,theta)R.*[cos((0:8)'/9*2*pi+theta)
sin((0:8)'/9*2*pi+theta)];
% 移动到目标位置的总距离函数（目标函数）
dif=@(R,theta)sum(sqrt(sum((now-aim(R,theta)).^2,2)))
dif2=@(x)sum(sqrt(sum((now-aim(x(1),x(2))).^2,2)))
% 求区间内最小值点，并绘制求解过程
options = optimset('PlotFcns',@optimplotfval,'TolX',1e-9,'Display','iter');
y=fminsearch(dif2,[105,0],options);
fprintf('rho = %f (m)\n',y(1))
fprintf('theta = %f (rad)\ntheta = %f (°)\n',y(2),y(2)*180/pi)
% 绘制目标函数的 3D 图会标出最小值点
figure
ax=fsurf(dif,[98 112 -.3*pi/180 .3*pi/180]);
ax.LineStyle='none';
colorbar
hold on
scatter3(y(1),y(2),dif2(y),'filled','r')

```

问题一 (3) -2

```

%% 问题一(3)
%% 每个无人机迭代计算并依照计算结果调整自身位置
% 尝试多种迭代步长，如固定步长、指数衰减步长等

clc, clear, close all, format short g
% 预处理标准位置
global R
R=100;
R=104.849649877396;
theta=-0.0215687329211256*pi/180;
std_rho=[0; ones(9,1)*R];
std_theta=[0; (0:8)'/9*2*pi]+theta;
global now_pos std_pos
std_pos=std_rho.*[cos(std_theta) sin(std_theta)];
now_pos=std_pos;
% 预设图窗大小
set(gcf, 'Position', [0 0 800 800]);

color=["#0072BD"
       "#D95319"
       "#EDB120"
       "#7E2F8E"
       "#77AC30"
       "#4DBEEE"
       "#A2142F"];

tic
% 预处理重复次数，5000 次用于调试，约 2 秒出结果；50000 次提高精度
% 实际使用时应当保存预处理结果重复利用
ts=500;
% 预分配内存以获得更高的运算速度
angles=zeros(ts,2);
% 分别计算四种情况（另外四种可以由对称性类似求解）
data=zeros(4,ts*7,2);
for self=2:5
    subplot(220+self-1)
    title(['self = ' num2str(self) ' (' num2str(11-self) ' is
similar)'])
    xlabel('smallest degree(°)')
    ylabel('second smallest degree(°)')

```

```

axis([0 90 0 90])
hold on
l = [];
% 枚举选取的第三架无人机
for third=2:9
    % 选择的第三架无人机不能为自己
    if self==third
        continue
    end
    % 蒙特卡洛模拟，反复随机自身的位置
    for test=1:ts
        % 径向偏移区间为[-2, 12]
        error_rho=14*rand-2;
        % 角度偏移区间为[-0.3°, 0.3°]
        error_theta=0.3*pi/180*(2*rand-1);
        % 更新自身偏移
        now_pos=std_pos;
        self_rho=std_rho(self+1)+error_rho;
        self_theta=std_theta(self+1)+error_theta;
        now_pos(self+1,:)=self_rho.*[cos(self_theta)
sin(self_theta)];
        % 获取自身测量所得的三个角度值
        angle=[getangle(self,0,1) getangle(self,0,third)
getangle(self,1,third)];
        angle=sort(angle);
        % 检测两个小角和等于大角
        if abs(angle(1)+angle(2)-angle(3))>1e-9
            error('add')
        end
        % 保留两个小角
        angles(test,:)=angle(1:2);
    end
    angles=angles*180/pi;
    % 选择的第三架无人机不能为自己，重新标号
    id=third-1-boolean(third>self);
    data(self-1,(id-1)*ts+1:id*ts,:)=angles;
    % 绘制自身位置的不同偏移情况时，所测量的角度的两个较小值的分布情况
    plot(angles(:,1),angles(:,2),'.','MarkerSize',1,'Color'
,color(id))

```

```

        % 设置图例，并放大点以便于观察颜色
        l = [1
plot(NaN,NaN, '.', 'MarkerSize',10, 'Color',color(id), 'DisplayNam
e',sprintf("FY%02d",third))];
        end
        legend(l)
end
toc
% 保存为 pdf 矢量图
if ts >= 50000
    exportgraphics(gcf,'pic_hold.pdf','ContentType','vector')
    exportgraphics(gcf,'pic_hold.png','Resolution',600)
end
tic
% R=100;
map=[0 1 2 3 4 4 3 2 1];
% figure
% 初始化
% polarscatter(now_theta,now_rho)
% hold on
% polarplot(linspace(0,2*pi),linspace(R,R))
ts2=2000;
% lambda=3e-1;
for base=1:-0.1:0.1
    now_rho=[0 100,98,112,105,98,112,105,98,112]';
    now_theta=[0
0,40.10,80.21,119.75,159.86,199.96,240.07,280.17,320.28]'/180*
pi;
    now_pos=now_rho.*[cos(now_theta) sin(now_theta)]
    differ=zeros(ts2,10);
    movs=zeros(ts2,1);
    mov=0;
    lambda=base;
    for test=1:ts2
        if max(sqrt(sum(now_pos.^2,2)))>150
            error('out of range')
            break
        end
        differ(test,:)=sqrt(sum((now_pos-std_pos).^2,2));
        % lambda = 1-(test-1)/ts2/2;
        % lambda=base^-test;

```

```

        dis=sum((now_pos-std_pos).^2,2);
        [~,I]=sort(dis);
        second=I(2)-1;
        third=I(3)-1;
        fprintf('选择的第二架发射信号的无人机编号:
FY%02d\n',second);
        fprintf('选择的第三架发射信号的无人机编号:
FY%02d\n',third);
        for self=1:9
            if self==second || self==third
                continue
            end
            fprintf('被动接收信号的无人机编号: FY%02d\n',self);
            % 假设发射信号的无人机的位置是正确的, 求解自身位置
            calc_pos=get_self_pos(self,second,third);
            % 对比误差并作图
            % now_pos(self+1,:)
            my_error=norm(calc_pos-now_pos(self+1,:));
            %
            [theta,
rho]=cart2pol(casemilogylc_pos(1),calc_pos(2));
            %
            polarscatter(theta,rho)
            if my_error>1e-9
                % 由于没有准确的发射机, 所以不可能得到绝对准确的结果
                % warning('position error=%g',my_error)
            end
            % 尝试移动
            calc_mov=std_pos(self+1,:)-calc_pos;
            now_pos(self+1,:)=now_pos(self+1,:)+lambda*calc_mov
;
            mov=mov+lambda*norm(calc_mov);
        end
        movs(test)=mov;
    end
    toc
    figure
    % differ=cumsum(differ,2);
    loglog(sum(differ,2)/9,'LineWidth',2,'DisplayName','平均误差')
    hold on
    % yyaxis left

```

```

    for i=1:9
        loglog(differ(:,i+1),'DisplayName',sprintf('误差
FY%02d',i))
    end
    legend
    ylabel('定位误差（米）')
    yyaxis right
    loglog(movs,'DisplayName','累计移动距离','LineWidth',2)
    title(sprintf('\lambda=%.2f^{-迭代次数}',base))
    xlabel('迭代次数')
    ylabel('累计移动距离（米）')
    % 导出结果到文件
    % exportgraphics(gcf,sprintf('optimization_%.2f^迭代次数%d_
平均误差%g_累计移动距
离%g.png',base,ts2,sum(differ(ts2,:))/9,movs(end)),'Resolution'
,600)
    exportgraphics(gcf,sprintf('optimization_迭代次
数%d_lambda=%.2f_平均误差%g_累计移动距
离%g.png',ts2,lambda,sum(differ(ts2,:))/9,movs(end)),'Resolutio
n',600)
    % exportgraphics(gcf,sprintf('optimization_1-%gtest_平均误
差%g_累计移动距
离%g.png',lambda,sum(differ(ts2,:))/9,movs(end)),'Resolution',6
00)
    close
end

function ret=getstdir(self,x,y)
    % 使正弦定理计算标准角度方向
    global std_pos
    a=std_pos(x+1,:)-std_pos(self+1,:);
    b=std_pos(y+1,:)-std_pos(self+1,:);
    ret=a(1)*b(2)-a(2)*b(1);
end

function ret=getangle(self,x,y)
    % 使余弦定理计算应该测量得到的角度值
    global now_pos
    a=norm(now_pos(x+1,:)-now_pos(self+1,:));
    b=norm(now_pos(y+1,:)-now_pos(self+1,:));

```

```

c=norm(now_pos(x+1,:)-now_pos(y+1,:));
ret=acos((a^2+b^2-c^2)/(2*a*b));
end

function calc_pos=get_self_pos(self,second,third)
    global R
    % 无人机知道角度和发射信号的无人机编号对应
    angle=[getangle(self,0,second) getangle(self,0,third)
getangle(self,second,third)];
    % 【∠OPA ∠BPO ∠BPA】
    dir=([getstdir(self,0,second) getstdir(self,third,0)
getstdir(self,third,second)]>0)*2-1;
    beta=angle.*dir;
    if abs(beta(1)+beta(2)-beta(3))>1e-9
        error('directed angle, %g',abs(beta(1)+beta(2)-
beta(3)))
    end
    % 变换坐标系使第二架无人机（second）在新横轴上
    rot=second-1;
    arti_third=third-rot;
    if arti_third<1
        arti_third=arti_third+9;
    end
    % arti_third
    alpha=40*(double(arti_third)-1)*pi/180;
    gama=atan((sin(beta(1)) * (sin(alpha + beta(2)) -
sin(beta(2)))) / (sin(beta(1)) * cos(alpha + beta(2)) +
cos(beta(1)) * sin(beta(2))));
    my_error=sin(beta(1))*sin(alpha+beta(2)-gama)-
sin(beta(2))*sin(gama+beta(1));
    if my_error>1e-9
        error('equation error=%g',my_error)
    end
    OP=R*sin(gama+beta(1))/sin(beta(1));
    % 变换坐标系回原来的坐标系
    real_gama=gama+40*double(rot)*pi/180;
    calc_pos=OP*[cos(real_gama) sin(real_gama)];
end

```

问题一（3）-3

```

%% 问题一(3)
%% 每个无人机迭代计算并依照计算结果调整自身位置
% 不使用距离信息
% 每次调整以测量到的角度信息的某种关系为终止条件
% 每次调整的方向，沿某个信号的发射方向，或垂直某个信号的发射方向
% 发射信号的无人机依次轮转
% 接收信号的无人机，依据自身接收到的角度信息，判断是否轮到自己调整位置
% 判断方式：接收到的最小的两个角度均小于  $15^\circ$ ，则轮到自己调整位置
% if 当前调整方式==径向调整
% // 使得最大的角度与理想位置的角度相等( $20^\circ$ )
% call 径向调整到理想位置
% if 当前调整方式==切向调整
% // 使得较小的其他角度大小相等
% call 切向调整到理想位置

% 结果在机器精度中能够收敛

clc, clear, close all, format long g

global now_rho now_theta
now_rho=[0 100,98,112,105,98,112,105,98,112]
now_theta=[0
0,40.10,80.21,119.75,159.86,199.96,240.07,280.17,320.28]/180*
pi;

tic
ts=400;
run=zeros(9,1);
run_round=zeros(ts,1);
rho_std=zeros(ts,1);
theta_degree_std=zeros(ts,1);
for test=1:ts
    % 每轮第一步径向调整，第二步切向调整
    fprintf('第%d轮\n',test)
    rho_std(test)=std(now_rho(2:10));
    theta_degree_std(test)=std(mod(now_theta(2:10)*180/pi+20,40
));

```

```

for part=1:2
    for self=1:9
        % 每次发送信息时，由相邻两个无人机(send)发射信号
        % 其余所有无人机接收到信号时，将接收到的角度的较小两个值
        % 与 15°进行比较，若两个值都小于 15°，说明轮到此无人机(self)进行调整
        % 由于正文中已经证明对应关系唯一，为了加快代码运行效率，
        % 此代码中 self 与 send 的编号直接选定
        send=[4 5]+self;
        send=send-(send>9)*9;
        if part==1
            % 径向调整，使得接收到的三个角度值的最大值变成 20 度
            % [rho / 2, rho * 2]
            begin_rho=now_rho(self+1);
            l=now_rho(self+1)/2;
            r=now_rho(self+1)*2;
            % 二分算法求解单调函数
            while r-l>1e-12
                now_rho(self+1)=(l+r)/2;
                if getangle(self,send(1),send(2))*180/pi<20
                    r=now_rho(self+1);
                else
                    l=now_rho(self+1);
                end
            end
            now_rho(self+1)=(l+r)/2;
            fprintf('无人机 FY%02d 径向调整后测量得到的较大角
            =%f°\n',self,getangle(self,send(1),send(2))*180/pi)
            % 认为一次调整为单向直线运动
            run(self)=run(self)+abs(now_rho(self+1)-
            begin_rho);
            run_round(test)=run_round(test)+abs(now_rho(self+1)-begin_rho);
        else
            % 切向调整，使得接收到的三个角度值的较小两个相等
            % [theta - 1°, theta + 1°]
            begin_theta=now_theta(self+1);
            l=now_theta(self+1)-pi/180;
            r=now_theta(self+1)+pi/180;

```

```

        % 二分算法求解单调函数
        while r-l>1e-14
            now_theta(self+1)=(l+r)/2;
            if
getangle(self,0,send(1))*180/pi>getangle(self,0,send(2))*180/p
i
                r=now_theta(self+1);
            else
                l=now_theta(self+1);
            end
        end
        now_theta(self+1)=(l+r)/2;
        final1=getangle(self,0,send(1))*180/pi;
        final2=getangle(self,0,send(2))*180/pi;
        fprintf('无人机 FY%02d 切向调整后测量得到的两个较小
角=%f°, %f°\n',[self final1 final2])
        % 认为一次调整为弦/圆弧上运动，由于运动角度很小，可
        以认为两者相等
        run(self)=run(self)+abs(now_theta(self+1)-
begin_theta)*now_rho(self+1);
        run_round(test)=run_round(test)+abs(now_theta(s
elf+1)-begin_theta)*now_rho(self+1);
    end
end
end
% now_rho
% degree=now_theta*180/pi
end
% 检验模型实际效果，并绘制可视化图形
polarscatter(now_theta,now_rho)
now_rho
fprintf('rho 标准差%g（不含 FY00）\n',std(now_rho(2:10)))
uni_degree=mod(now_theta*180/pi+20,40)
fprintf('uni_degree 标准差%g（不含 FY00）
\n',std(uni_degree(2:10)))
fprintf('%d 次迭代\n',ts)
run
fprintf('平均路程%f (m)\n',mean(run))
toc

```

```

semilogy(1:ts,run_round,'LineWidth',1)
xlabel('调整次数')
ylabel('单次调整距离（米）')
yyaxis right
plot(1:ts,cumsum(run_round)/9,'LineWidth',1)
ylabel('平均累计调整距离（米）')
figure
semilogy(rho_std,'LineWidth',1)
xlabel('调整次数')
ylabel('径向标准差（米）')
yyaxis right
semilogy(theta_degree_std,'LineWidth',1)
ylabel('切向标准差（度）')

function ret=getangle(self,x,y)
    % 使余弦定理计算应该测量得到的角度值
    global now_rho now_theta
    now_pos=now_rho.*[cos(now_theta) sin(now_theta)];
    a=norm(now_pos(x+1,:)-now_pos(self+1,:));
    b=norm(now_pos(y+1,:)-now_pos(self+1,:));
    c=norm(now_pos(x+1,:)-now_pos(y+1,:));
    ret=acos((a^2+b^2-c^2)/(2*a*b));
end

```

问题二

```

%% 问题 2
%% 每个无人机迭代计算并依照计算结果调整自身位置
% 不使用距离信息
% 每次调整以测量到的角度信息的某种关系为终止条件
% 每次调整的方向，沿某个信号的发射方向，或垂直某个信号的发射方向
% 发射信号的无人机依次轮转（共三组，每组三架无人机）
% 接收信号的无人机，依据自身接收到的角度信息，判断是否轮到自己调整位置

%      if 当前调整方式==径向调整
%          // 使得最大的角度与理想位置的角度相等
%          call 径向调整到理想位置
%      if 当前调整方式==切向调整
%          // 使得较小的其他角度与标准位置的比例相等
%          call 切向调整到理想位置

% 结果在机器精度中能够收敛
clc, clear, close all, format long g

global std_pos now_pos
%% 斜坐标系, (i, j) = 直角坐标系(i*a+j*b*cos(0), j*b*sin(0))
% 斜坐标系上，无人机标准的位置的坐标值在两个轴上分别为等差数列
% 有利于检验模型实际效果
std_new_pos=[
    4 0
    3 1
    3 0
    2 2
    2 1
    2 0
    1 3
    1 2
    1 1
    1 0
    0 4
    0 3
    0 2
    0 1
    0 0];

```

```

a=90;
b=110;
O=60*pi/180;
vec_i=[1 0];
vec_j=[cos(O) sin(O)];
std_pos=std_new_pos(:,1)*a*vec_i+std_new_pos(:,2)*b*vec_j;

% 预处理标准角度值，减小重复运算
std_angle=zeros(15,15,15);
for self=1:15
    for second=1:15
        if second==self
            continue
        end
        for third=1:15
            if third==second || third==self
                continue
            end
            std_angle(self,second,third)=get_std_angle(self,second,third);
        end
    end
end

% 略有偏差
now_new_pos=std_new_pos+0.001*(rand(15,2)*2-1);
now_pos=now_new_pos(:,1)*a*vec_i+now_new_pos(:,2)*b*vec_j;
my_error=std(mod(now_new_pos+0.5,1))
RATIO = 0.1;

ts=1;
for test=1:ts
    for emit_group=1:3
        if emit_group==1
            % 发射机的顺序在接收机来看为逆时针
            emit=[14 15 10];
            work=[1 2 4 7 5];
        elseif emit_group==2
            emit=[3 1 2];
            work=[11 12 13 14 8]
        end
    end
end

```

```

elseif emit_group==3
    emit=[7 11 12];
    work=[15 10 6 3 9];
end
for part=1:2
    for self=1:15
        if ~ismember(self,work)
            continue
            warning('self=%d',self)
        end
        if part==1
            % 径向调整,使得接收到的三个角度值的最大值变成
20 度
            %  $\pm \text{RATIO} * \rho$ 
            vec=now_pos(self,:)-now_pos(emit(2),:);
            [theta, rho]=cart2pol(vec(1), vec(2));
            l=rho*(1-RATIO);
            r=rho*(1+RATIO);
            % 二分算法求解单调函数
            while r-l>1e-10
                mid=(l+r)/2;
                now_pos(self,:)=now_pos(emit(2),:)+mid.*
[cos(theta) sin(theta)];
                tmp_angle=get_angle(self,emit(1),emit(3))
);
                if
tmp_angle<std_angle(self,emit(1),emit(3))
                    r=mid;
                else
                    l=mid;
                end
            end
            mid=(l+r)/2;
            now_pos(self,:)=now_pos(emit(2),:)+mid.*[co
s(theta) sin(theta)];
            fprintf('径向角度误
差%g\n',get_angle(self,emit(1),emit(3))-
std_angle(self,emit(1),emit(3)))
        else
            % 切向调整,使得接收到的三个角度值的较小两个相等

```

```

        % [theta - 1°, theta + 1°]
        vec=now_pos(self,:)-now_pos(emit(2),:);
        [theta, rho]=cart2pol(vec(1), vec(2));
        l=-1*pi/180;
        r=1*pi/180;
        aim_angle=[std_angle(self,emit(1),emit(2))
std_angle(self,emit(2),emit(3))
std_angle(self,emit(1),emit(3))];
        % 格式化有向角
        if aim_angle(1)>aim_angle(3)
            aim_angle(1)=aim_angle(3);
        end
        % 二分算法求解单调函数
        while r-l>1e-10
            mid=(l+r)/2;
            now_pos(self,:)=now_pos(emit(2),:)+rho.*
[cos(theta+mid) sin(theta+mid)];
            tmp_angle=[get_angle(self,emit(1),emit(2))
get_angle(self,emit(2),emit(3))
get_angle(self,emit(1),emit(3))];
            if tmp_angle(1)<tmp_angle(2)
                tmp_angle(1)=tmp_angle(3)-
tmp_angle(2);
            end
            if abs(tmp_angle(1)/tmp_angle(3)-
aim_angle(1)/aim_angle(3))<1e-6
                break
            elseif
tmp_angle(1)/tmp_angle(3)<aim_angle(1)/aim_angle(3)
                r=mid;
            else
                l=mid;
            end
        end
        tmp_angle=[get_angle(self,emit(1),emit(2))
get_angle(self,emit(2),emit(3))
get_angle(self,emit(1),emit(3))];
        mid=(l+r)/2;
        now_pos(self,:)=now_pos(emit(2),:)+rho.*[co
s(theta+mid) sin(theta+mid)];

```

```

        fprintf('切向角度误差%g\n',tmp_angle(1)/tmp_angle(3)-aim_angle(1)/aim_angle(3))
    end
end
end
% 检验模型实际效果
[theta, rho]=cart2pol(now_pos(:,1),now_pos(:,2));
polarscatter(theta,rho)
now_new_pos=rho./sin(0).*[sin(0-theta) sin(theta)]./[a
b]

my_error=std(mod(now_new_pos+0.5,1));
fprintf('标准差为%g, %g（单位无人机间隔）\n',my_error)
end
end

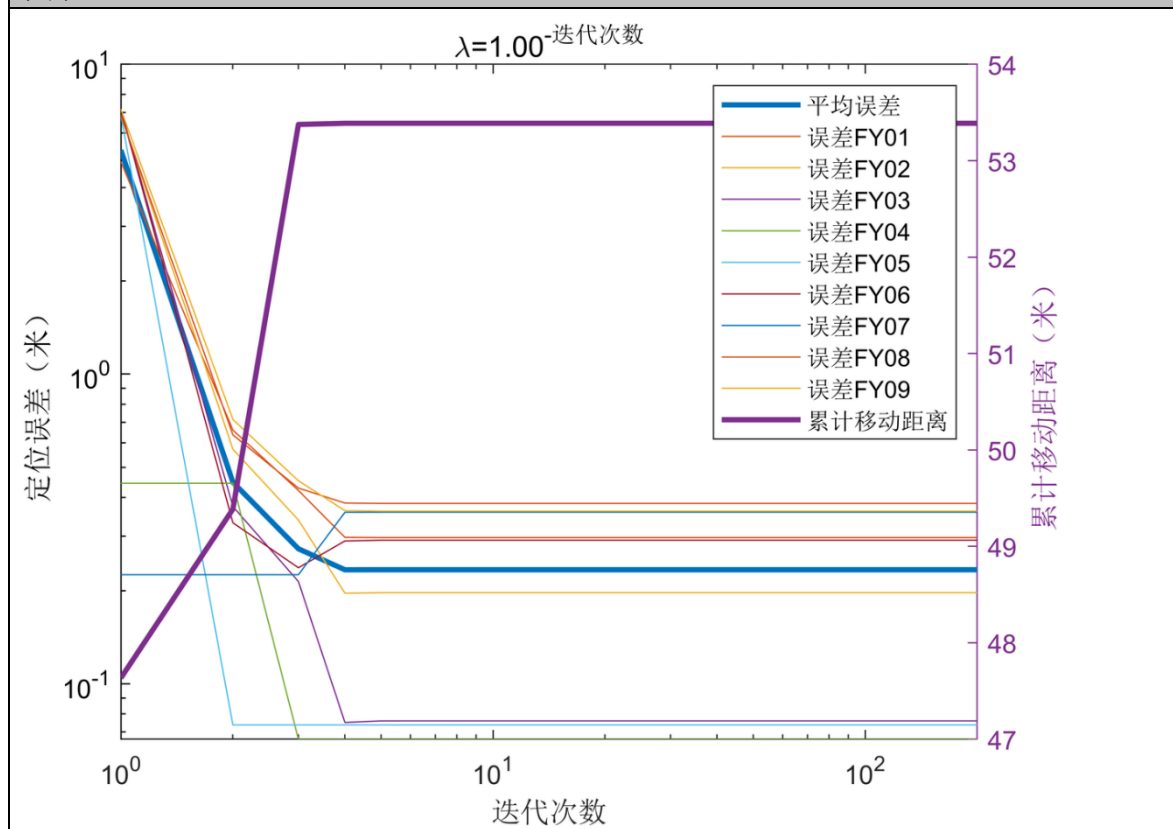
function ret=get_std_angle(self,x,y)
% 使余弦定理计算应该测量得到的角度值
global std_pos
i=norm(std_pos(x,:)-std_pos(self,:));
j=norm(std_pos(y,:)-std_pos(self,:));
k=norm(std_pos(x,:)-std_pos(y,:));
ret=real(acos((i^2+j^2-k^2)/(2*i*j)));
if ret<1e-6
    ret=0;
end
end

function ret=get_angle(self,x,y)
% 使余弦定理计算应该测量得到的角度值
global now_pos
i=norm(now_pos(x,:)-now_pos(self,:));
j=norm(now_pos(y,:)-now_pos(self,:));
k=norm(now_pos(x,:)-now_pos(y,:));
ret=real(acos((i^2+j^2-k^2)/(2*i*j)));
end

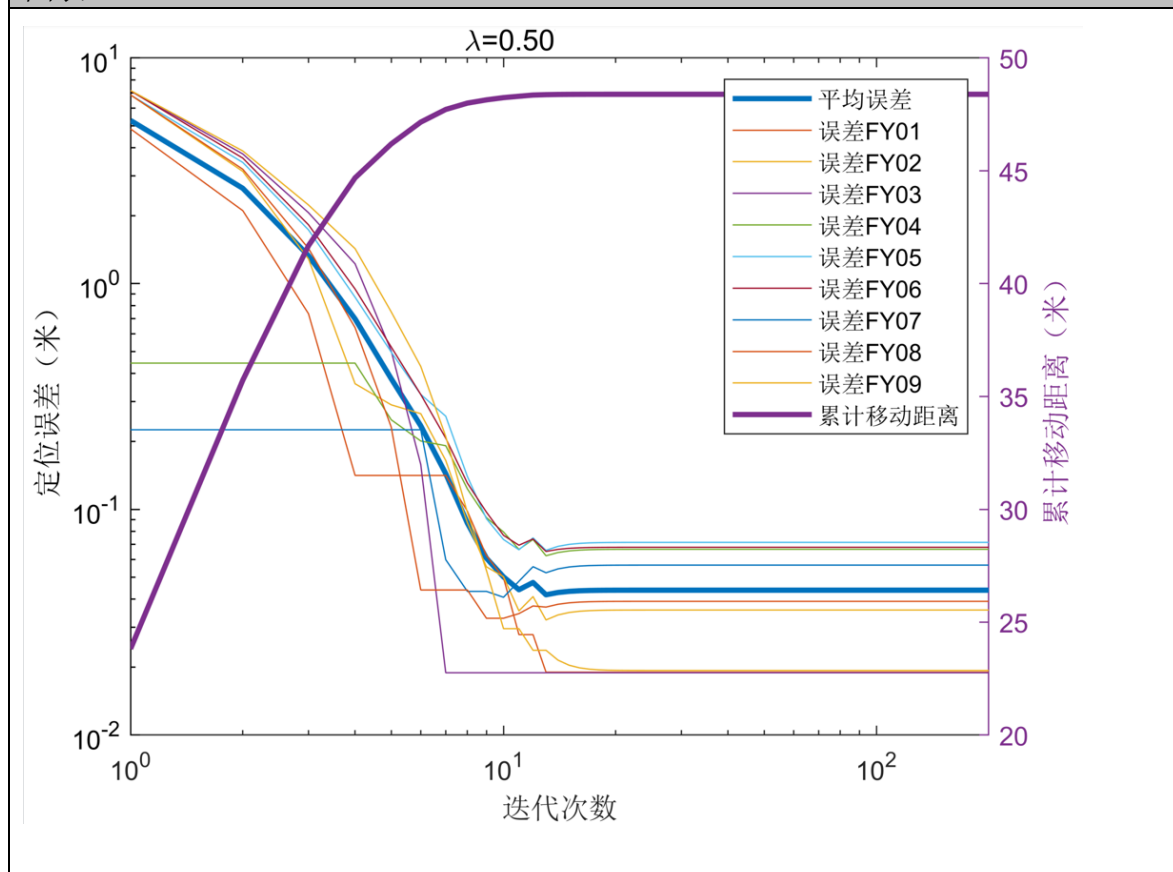
```

附录 2_图表

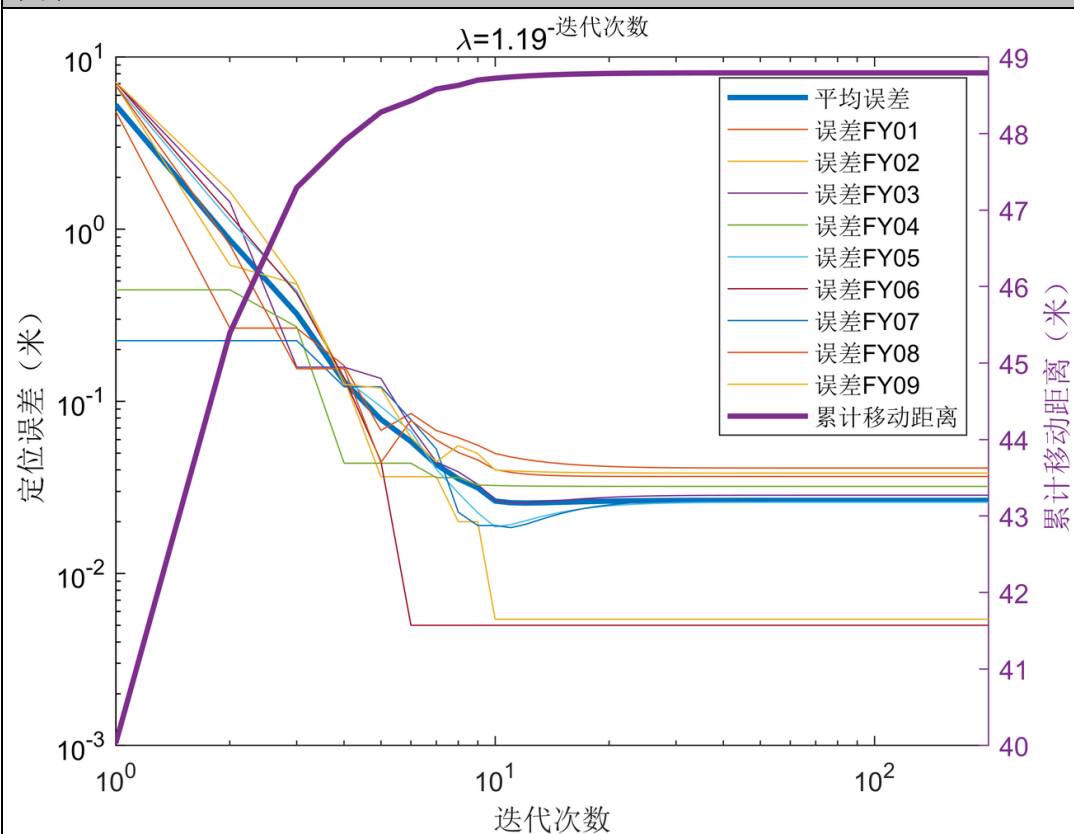
图表 1



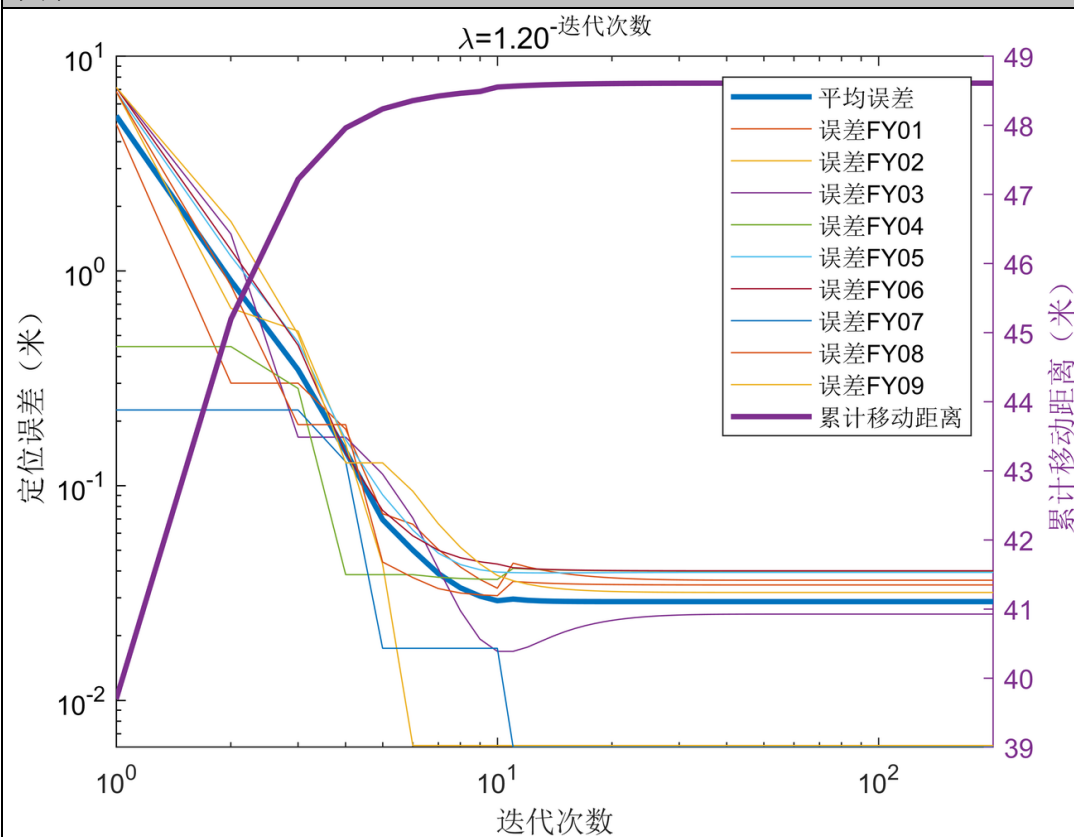
图表 2



图表 3



图表 4



附录 3_表格

表格 1

FY03'	6. 04E-14	' FY02'	0
FY04'	3. 55E-14	' FY04'	9. 53E-14
FY05'	5. 18E-13	' FY05'	2. 17E-13
FY06'	6. 55E-13	' FY06'	1. 71E-13
FY07'	5. 86E-14	' FY07'	1. 56E-13
FY08'	6. 36E-14	' FY08'	3. 28E-13
FY09'	4. 49E-14	' FY09'	2. 84E-14
' FY02'	2. 01E-14	' FY02'	2. 84E-14
' FY03'	3. 18E-14	' FY03'	3. 78E-14
' FY05'	1. 15E-13	' FY04'	9. 10E-14
' FY06'	4. 26E-14	' FY06'	4. 55E-14
' FY07'	1. 42E-14	' FY07'	1. 42E-14
' FY08'	7. 98E-14	' FY08'	4. 39E-14
' FY09'	1. 42E-14	' FY09'	9. 10E-14

表格 2

λ decrease	稳定后的平均误差/m	所有无人机的移动距离之和/m
1	0. 233722	53. 3887
1. 01	0. 0659997	55. 1606
1. 02	0. 205801	59. 137
1. 03	0. 0376664	54. 9065
1. 04	0. 0479391	53. 2287
1. 05	0. 0490965	52. 586
1. 06	0. 0528469	50. 1272
1. 07	0. 0369355	49. 8156
1. 08	0. 0451638	49. 9423
1. 09	0. 0349162	50. 0655
1. 1	0. 0474589	53. 3221
1. 11	0. 0576372	51. 3493
1. 12	0. 0906235	48. 8849
1. 13	0. 0929101	49. 6619
1. 14	0. 0847036	48. 9165
1. 15	0. 0550282	48. 6643
1. 16	0. 0373521	48. 4883
1. 17	0. 0471432	48. 6583
1. 18	0. 0525909	49. 1226
1. 19	0. 0265913	48. 7947
1. 2	0. 0288387	48. 6113
1. 21	0. 027154	48. 6412
1. 22	0. 0282296	48. 6737
1. 23	0. 0352058	48. 688
1. 24	0. 0334312	48. 6883

1. 25	0. 0780933	48. 8476
1. 26	0. 0451152	49. 0294
1. 27	0. 061138	48. 7797
1. 28	0. 0516503	48. 6725
1. 29	0. 143688	49. 2127
1. 3	0. 17134	48. 7687
1. 31	0. 148262	48. 4582
1. 32	0. 172862	48. 1367
1. 33	0. 17686	47. 9499
1. 34	0. 160786	47. 6922
1. 35	0. 155046	47. 3044
1. 36	0. 160887	47. 136
1. 37	0. 17246	47. 0446
1. 38	0. 185262	46. 8024
1. 39	0. 200158	46. 6516
1. 4	0. 21988	46. 6065
表格 3		
λ const	稳定后的平均误差/m	所有无人机的移动距离之和/m
0. 01	0. 744168	41. 2511
0. 02	0. 128858	48. 5907
0. 03	0. 0245478	49. 6929
0. 04	0. 0248628	50. 2728
0. 05	0. 0297843	50. 2376
0. 06	0. 0320062	49. 7521
0. 07	0. 0267562	49. 8112
0. 08	0. 0321506	49. 6295
0. 09	0. 0319662	49. 663
0. 1	0. 0326501	50. 1619
0. 11	0. 0328989	50. 4217
0. 12	0. 0280007	49. 8192
0. 13	0. 0294682	49. 0886
0. 14	0. 0267851	49. 957
0. 15	0. 0851854	49. 2341
0. 16	0. 033567	49. 2036
0. 17	0. 0929049	49. 6933
0. 18	0. 0911257	49. 3466
0. 19	0. 0412119	49. 9209
0. 2	0. 0294039	49. 5561
0. 21	0. 0679178	50. 2476
0. 22	0. 0278987	48. 8913
0. 23	0. 0330952	50. 8428
0. 24	0. 102174	49. 4614
0. 25	0. 0330394	48. 915
0. 26	0. 0270462	53. 1854

0. 27	0. 0330681	48. 9815
0. 28	0. 0432855	49. 3645
0. 29	0. 0262767	48. 8185
0. 3	0. 0446362	48. 854
0. 31	0. 041824	51. 9647
0. 32	0. 0330862	49. 0953
0. 33	0. 0371636	48. 735
0. 34	0. 0403453	48. 9452
0. 35	0. 0621273	48. 6941
0. 36	0. 0325589	55. 176
0. 37	0. 0591587	55. 0432
0. 38	0. 0363122	48. 4483
0. 39	0. 0387922	48. 6718
0. 4	0. 0930119	49. 482
0. 41	0. 0839858	49. 8141
0. 42	0. 101819	48. 8359
0. 43	0. 0845943	48. 5139
0. 44	0. 0467157	49. 6183
0. 45	0. 0449048	48. 9145
0. 46	0. 0403521	48. 6833
0. 47	0. 0414273	49. 243
0. 48	0. 0442197	48. 6746
0. 49	0. 0512978	49. 0555
0. 5	0. 0438382	48. 3907
0. 51	0. 103885	48. 2631
0. 52	0. 046374	51. 1616
0. 53	0. 0336966	53. 0195
0. 54	0. 0345331	52. 0073
0. 55	0. 0399829	51. 5968
0. 56	0. 090977	49. 4566
0. 57	0. 0644566	50. 6205
0. 58	0. 0312842	52. 1185
0. 59	0. 36059	53. 4825
0. 6	0. 0354721	53. 0904
0. 61	0. 281616	54. 9169
0. 62	0. 0633557	48. 5755
0. 63	0. 0666718	48. 4454
0. 64	0. 0693916	48. 4388
0. 65	0. 0942638	48. 5845
0. 66	0. 11454	48. 4182
0. 67	0. 122024	48. 2528
0. 68	0. 0847715	51. 6826
0. 69	0. 0286181	51. 5349
0. 7	0. 0510905	51. 9111

0.71	0.0411069	49.6557
0.72	0.053261	49.3742
0.73	0.0959995	50.1957
0.74	0.0578854	51.2412
0.75	0.0393291	50.3887
0.76	0.0344226	50.9036
0.77	0.0432006	51.5025
0.78	0.0345735	51.1779
0.79	0.0465721	51.3891
0.8	0.0463326	53.5342
0.81	0.0375115	52.7141
0.82	0.0718068	55.4812
0.83	0.023471	49.8953
0.84	0.072769	50.7859
0.85	0.0567672	50.5332
0.86	0.0434911	51.0467
0.87	0.13383	49.5645
0.88	0.0920721	50.5277
0.89	0.0375596	50.7829
0.9	0.0630581	52.6693
0.91	0.0852623	55.7995
0.92	0.0484852	49.6237
0.93	0.0566848	49.995
0.94	0.0372709	50.6478
0.95	0.0472788	52.445
0.96	0.159127	52.6169
0.97	0.0328404	55.6884
0.98	0.0551089	64.4004
0.99	0.0459221	57.125
1	0.233722	53.3887

附录 4_算法

算法 1: *Nelder – Mead*单纯形算法

a. 生成单纯形：设定初始点为 $X = (105, 0)$ ，围绕初始点 X 添加各分量值的 5%构造单纯形。

b. 根据如下过程反复修改单纯形：

i. 排序：按最小函数值 $L(x(1))$ 到最大函数值 $L(x(n+1))$ 的顺序对单纯形中的点进行排序。

ii. 计算反射点： $r = 2m - x(n+1)$, $m = \frac{\sum_{i=1}^n x(i)}{n}$ 。

3. 如果 $L(x(1)) \leq L(r) < L(x(n))$ ，则接受 r 并停止迭代。

2. 如果 $L(r) < L(x(1))$ ，则计算延伸点 s , $s = m + 2(m - x(n+1))$ 。

a. 如果 $L(s) < L(r)$ ，接受 s 并停止迭代。

b. 否则，接受 r 并停止迭代。

3. 如果 $L(r) \geq L(x(n))$ ，则在 m 和 $x(n+1)$ 或 r （取目标函数值较低者）之间执行。

a. 如果 $L(r) < L(x(n+1))$ （即 r 优于 $x(n+1)$ ），则计算 $c = m + \frac{(r-m)}{2}$ ，如果 $L(c) < L(r)$ ，则接受 c 并停止迭代，否则执行步骤 4。

b. 如果 $L(r) \geq L(x(n+1))$ ，则计算 $cc = m + \frac{(x(n+1)-m)}{2}$ ，如果 $L(cc) < L(x(n+1))$ ，则接受 cc 并终止迭代，否则，继续执行步骤 4。

计算迭代的单纯形 $v(i) = x(1) + \frac{(x(i)-x(1))}{2}$ ，符合停止条件时停止运行。